

PYTHON MASTER NOTES

Part 1

Foundations & Core Basics

For college exams • coding interviews • competitive programming
software engineering • AI / ML

How to read the boxes in these notes

Marker	Meaning
★ MUST KNOW	Core fact. If you forget this, everything built on it becomes shaky.
⊙ INTERVIEW	A point interviewers and exam-setters love to ask.
⚠ COMMON MISTAKE	An error beginners (and tired experts) make again and again.
⚡ SHORTCUT	A faster / more Pythonic way to do the same thing.
◆ DEEP CONCEPT	What is really happening under the hood. Read this after the basics feel easy.

All examples were run on **Python 3.12**. Every "Output" block was produced by actually running the code (programs that use input() show the keyboard input that was given).

Contents

How to use these notes	4
1. Meet Python	5
1.1 What is Python?	5
1.2 Features of Python	6
1.3 The Python Interpreter	6
1.4 Interpreter vs Compiler	7
1.5 High-Level Language	8
1.6 Dynamic Typing	8
2. Variables, Objects & Memory	10
2.1 Variables and objects	10
2.2 Names share objects: aliasing	11
2.3 Mutable vs immutable objects	11
2.4 Python memory and object concept	12
2.5 Working with variables	13
3. Basic Syntax	14
3.1 Identifiers	14
3.2 Keywords	15
3.3 Comments	15
3.4 Basic syntax rules	16
3.5 Indentation	16
4. Data Types	18
4.1 What is a data type?	18
4.2 int (integers)	18
4.3 float (decimal numbers)	19
4.4 complex	20
4.5 bool	20
4.6 str (strings)	20
4.7 list	22
4.8 tuple	24
4.9 set	24
4.10 dict (dictionary)	25
4.11 list vs tuple vs set vs dict	26
4.12 None	26
4.13 Type checking: type() and isinstance()	27
5. Type Conversion	29
5.1 What is type conversion?	29
5.2 Implicit conversion	29
5.3 Explicit conversion	30
6. Input and Output	33
6.1 Input with input()	33

6.2 Output with print()	34
7. Operators	36
7.1 What are operators?	36
7.2 Arithmetic operators	36
7.3 Comparison (relational) operators	37
7.4 Logical operators	38
7.5 Assignment operators	39
7.6 Membership operators	40
7.7 Identity operators	41
7.8 == vs is (very important)	41
7.9 Bitwise operators	42
7.10 Operator precedence	43
8. Truthy and Falsy Values	45
9. Common Beginner Mistakes	47
10. Part 1 Quick Revision	48
11. Important Terminology	50
12. Twenty Conceptual Questions	52
13. Twenty Output-Based Questions	53
14. Twenty Practice Problems	55
15. Answers and Solutions	57
15.1 Conceptual answers	57
15.2 Output-based answers	58
15.3 Practice problem solutions	60
16. One-Page Cheat Sheet: Part 1	65

How to use these notes

Part 1 covers only the **foundations**: what Python is, how it runs, how names and objects work, the built-in data types, converting between types, input and output, and every kind of operator. Nothing here is "advanced", but a lot of it is asked in interviews because it exposes whether you truly understand Python or only copy syntax.

Suggested way to study

1. Read one section, then **close the PDF and type the examples yourself**. Do not copy-paste.
2. Before running any example, **predict the output** in your head. Then run it. Wrong predictions teach you the most.
3. Do the questions and practice problems at the end only after the sections feel comfortable.
4. Revise using the Quick Revision pages and the One-Page Cheat Sheet (last page).

Big picture of Part 1

Chapter	What you learn	Why it matters later
1. Meet Python	What Python is, features, interpreter, dynamic typing	Explains why Python behaves the way it does
2. Variables & Objects	Names, objects, references, id(), mutability	The root of most Python interview traps
3. Basic Syntax	Identifiers, keywords, comments, indentation	Writing code that even runs
4. Data Types	int, float, complex, bool, str, list, tuple, set, dict, None	Every DSA problem picks one of these
5. Type Conversion	Implicit vs explicit conversion	Input handling in CP and real programs
6. Input & Output	input(), print()	Reading test cases quickly
7. Operators	All 7 groups + precedence	Bit tricks, conditions, expressions
8. Truthy & Falsy	What counts as True or False	Cleaner conditions
9. Mistakes	Beginner errors and fixes	Save hours of debugging

1. Meet Python

1.1 What is Python?

Definition	Python is a high-level, general-purpose, interpreted, dynamically typed programming language. It was created by Guido van Rossum and first released in 1991 .
Why it exists	Guido wanted a language that is easy to read and write, so programmers spend time solving problems instead of fighting syntax. Python code often looks like plain English.
Name	Named after the British comedy group <i>Monty Python</i> , not the snake.
Used for	Web backends (Django, Flask, FastAPI), automation and scripting, data analysis (pandas, NumPy), AI / ML (PyTorch, TensorFlow, scikit-learn), competitive programming, testing, DevOps tools.

```
print("Hello, World!")
```

Output:
Hello, World!

The same program in C++ needs `#include <iostream>`, a `main()` function, `std::cout`, semicolons and braces. Python needs one line.

Ways to run Python

Way	How	Best for
REPL (interactive shell)	Type <code>python</code> (or <code>python3</code>) in a terminal. Prompt is <code>>>></code>	Testing one-liners, quick experiments
Script file	Save as <code>hello.py</code> , run <code>python hello.py</code>	Real programs
Notebook	Jupyter / Google Colab	Data science, AI/ML experiments
IDE / editor	VS Code, PyCharm	Bigger projects

★ MUST KNOW	Always use Python 3 . Python 2 reached end-of-life in January 2020. If you see <code>print "hello"</code> (without brackets) it is old Python 2 code.
--------------------	--

1.2 Features of Python

Feature	Simple meaning	Why it helps
Easy to learn & read	Clean, English-like syntax; indentation instead of braces	Fewer bugs, faster learning
Interpreted	Code is run by an interpreter (no separate manual compile step)	Write, run, fix quickly
High-level	You do not manage memory or hardware details	Focus on logic
Dynamically typed	No need to declare variable types	Less code
Object-oriented + multi-paradigm	Supports OOP, procedural and functional styles	Flexible design
Portable / cross-platform	Same code runs on Windows, Linux, macOS	Write once, run anywhere (with Python installed)
Huge standard library ("batteries included")	math, random, collections, heapq, re, os ...	Rarely reinvent the wheel
Huge third-party ecosystem	pip installs NumPy, pandas, PyTorch, Django ...	Best language for AI/ML
Open source & free	Anyone can use, modify and contribute	Huge community
Automatic memory management	Reference counting + garbage collector free unused objects	No malloc / free / delete
Extensible / embeddable	Can call C/C++ code and be embedded in other apps	NumPy and PyTorch are fast because of C/C++ inside

Limitations (be honest in interviews)

- **Slower than C/C++/Java** for raw CPU-heavy loops, because of dynamic typing and interpretation overhead. Heavy numeric work is done by libraries (NumPy, PyTorch) that run compiled C/C++/CUDA code.
- **GIL (Global Interpreter Lock)** in standard CPython: only one thread executes Python bytecode at a time, so CPU-bound multithreading does not scale. Use multiprocessing or native libraries.
- **Higher memory use** than low-level languages (everything is an object).
- Weak fit for mobile apps and very low-level system programming.

✂ SHORTCUT	For competitive programming, Python is slower than C++. Many judges accept PyPy , a faster Python implementation. Also read input with <code>sys.stdin</code> for large inputs (covered in a later part).
-------------------	--

1.3 The Python Interpreter

Definition	The interpreter is the program that reads your Python code and executes it. When you type <code>python file.py</code> , the interpreter starts, reads the file, and runs it.
Why it exists	Your CPU understands only machine code. Something must translate and run Python code. The interpreter does that for you, on any operating system.
Important term	CPython is the standard, official interpreter (written in C). When people say "Python", they usually mean CPython.

What really happens when you run a .py file? (a correction)

Many notes say "Python is interpreted, not compiled". That is only half true. In CPython, your source code is **first compiled to bytecode**, and then the bytecode is **executed by the Python Virtual Machine (PVM)**.

```
your_code.py --(1) compile--> bytecode (.pyc) --(2) PVM runs--> result
(source code)          (in __pycache__)    (interpreter loop)
```

- **Step 1 - Compile:** source text becomes **bytecode**, a compact, low-level instruction set for the PVM (not machine code). It may be cached as .pyc files inside a `__pycache__` folder so that the next import is faster.
- **Step 2 - Execute:** the **PVM** reads bytecode instructions one by one and performs them.

◆ DEEP CONCEPT	You can see bytecode with the <code>dis</code> module: <code>python -m dis file.py</code> . Bytecode is specific to the Python version, and it is not the same as machine code.
-----------------------	--

Python implementations

Implementation	Written in	Notes
CPython	C	Official, most common, what you install from python.org
PyPy	RPython	Uses JIT (just-in-time) compilation; often much faster for loops
Jython	Java	Runs on the JVM
IronPython	C#	Runs on .NET
MicroPython	C	Tiny Python for microcontrollers

◎ INTERVIEW	"Is Python compiled or interpreted?" Best answer: "Python source is compiled to bytecode, and the bytecode is interpreted by the Python Virtual Machine. So it is usually called an interpreted language, but strictly it does both."
--------------------	---

1.4 Interpreter vs Compiler

Compiler	Translates the whole program into machine code (an executable) before it runs. Examples: C, C++, Rust.
Interpreter	Reads and executes the program step by step (line by line or instruction by instruction). Examples: classic Python, JavaScript engines (with JIT), Ruby.

Point	Compiler	Interpreter
Translation	Entire program at once	One statement / instruction at a time
Output	Creates a separate executable file	No standalone executable is created (by default)
Speed of execution	Usually faster (already machine code)	Usually slower (translation during run)
Error reporting	Shows syntax errors for whole program before running	Runs until it reaches the error, then stops
Debugging	Harder to trace (separate build step)	Easier: run, see error, fix
Typical languages	C, C++, Go, Rust	Python, PHP, Ruby
Portability	Executable is tied to a CPU/OS	Same source works anywhere an interpreter exists

⚠ COMMON MISTAKE	Python still checks syntax for the whole file before running (compile-to-bytecode step). But runtime errors (like dividing by zero or using an undefined name) only appear when that line is reached.
-------------------------	---

```
print("This line runs first")
print(undefined_name) # NameError happens here, at runtime
```

Output:
This line runs first
NameError: name 'undefined_name' is not defined

1.5 High-Level Language

Definition	A high-level language is close to human language and hides hardware details such as CPU registers and memory addresses.
Why it exists	Writing machine code or assembly is slow and error-prone. High-level languages let you express ideas quickly and safely.

Level	Example	Closeness
Machine language	10110000 01100001	Only 0s and 1s, CPU understands directly
Assembly (low-level)	MOV AL, 61h	Human-readable machine instructions
Mid-level	C	Portable but manual memory management
High-level	Python, Java, JavaScript	Automatic memory, rich built-ins, readable

Example: sorting in Python is one line: `arr.sort()`. In a low-level language, you would write the sorting algorithm yourself and manage the memory.

🗣 INTERVIEW	High-level does not mean "better". It means more abstraction. You trade some speed and control for productivity.
--------------------	---

1.6 Dynamic Typing

Definition	In a dynamically typed language you do not declare the type of a variable. The type belongs to the object the variable refers to, and it is checked at runtime .
Why it exists	Less boilerplate, faster prototyping, flexible code.
Syntax	<code>x = 10</code> (no <code>int x</code>)

```
x = 10
print(type(x))
x = "hello"    # same name now points to a str object
print(type(x))
x = [1, 2, 3]
print(type(x))
```

Output:
<class 'int'>
<class 'str'>
<class 'list'>

The name `x` never had a fixed type. It was only a label sticking to different objects at different times.

Static vs dynamic typing

Point	Static typing (C++, Java)	Dynamic typing (Python)
Type declared?	Yes: <code>int x = 10;</code>	No: <code>x = 10</code>
Type is attached to	The variable	The object
Checked when?	At compile time	At runtime
Can a name change type?	No	Yes
Type errors found	Before running	Only when that line runs

Python is dynamic AND strong

```
print("5" + 5)
```

Output:
TypeError: can only concatenate str (not "int") to str

Term	Meaning	Python?
Dynamic typing	Types checked at runtime	Yes

Term	Meaning	Python?
Strong typing	Does not silently mix incompatible types	Yes ("5" + 5 is an error)
🕒 INTERVIEW	Dynamic typing is not the same as weak typing. Python is dynamically but strongly typed . JavaScript, by contrast, would give "55" for "5" + 5 (weak typing).	
⚠️ COMMON MISTAKE	Because types are not checked early, a typo like x = "10" and then x + 5 fails only when that line executes , possibly deep inside your program.	
💎 DEEP CONCEPT	Type hints such as x: int = 10 or def f(a: int) -> int: are optional documentation. Python does not enforce them at runtime. Tools like mypy check them before running.	

Duck typing (one-line idea)

"If it walks like a duck and quacks like a duck, it is a duck." Python cares about **what an object can do** (has len(), can be looped over) rather than what class it is. You will use this idea heavily later.

2. Variables, Objects & Memory

2.1 Variables and objects

Definition	In Python, a variable is a name that refers to an object . It is not a box that stores a value. The object lives in memory; the name is a label attached to it.
Why it exists	Everything in Python (numbers, strings, lists, functions) is an object. Names let you reach those objects without knowing where they live in memory.
Syntax	name = value (the = sign is called assignment or binding)

```
x = 10
name = "Parth"
print(x, name)
```

Output:
10 Parth

What `x = 10` really does, in order:

1. Python creates (or finds) an **int object** with value 10.
2. Python creates the **name** `x` in the current namespace.
3. Python **binds** the name `x` to that object.

Three things every object has

Property	Meaning	How to see it
Identity	A unique ID for the object (in CPython, its memory address). Never changes for the object's life.	<code>id(x)</code>
Type	What kind of object it is. Decides which operations are allowed.	<code>type(x)</code>
Value	The data it holds. May change only if the object is mutable .	<code>print(x)</code>

```
x = 10
print(type(x))
print(x)
print(id(x) == id(x))  # identity is stable
```

Output:
<class 'int'>
10
True

★ MUST KNOW	Variable = name (reference). Object = the actual data in memory. Assignment never copies an object, it only makes a name point to it.
--------------------	--

Terminology

Term	Simple meaning
Variable / name / identifier	A label used in code
Object	A piece of data in memory with identity, type and value
Reference	The link from a name to an object
Assignment / binding	Making a name point to an object
Rebinding	Making the same name point to a different object

2.2 Names share objects: aliasing

```
a = [1, 2, 3]
b = a          # NO copy. b is another name for the SAME list
b.append(4)
print(a)
print(a is b)
```

Output:
[1, 2, 3, 4]
True

`b = a` means: "make `b` point to whatever `a` points to." Both names now refer to one list, so changing it through `b` is visible through `a`.

⚠ COMMON MISTAKE	Beginners think <code>b = a</code> copies the list. It does not . For a real copy use <code>b = a.copy()</code> (shallow) or <code>copy.deepcopy(a)</code> (deep). Copying is covered fully in a later part.
-------------------------	---

```
x = 10
y = x          # y points to the same int 10
y = 20         # y is REBOUND to another object; x is unaffected
print(x, y)
```

Output:
10 20

Why did the int example behave differently? Because integers are **immutable**: you cannot change the number 10 into 20, you can only point the name somewhere else. Lists are **mutable**: they can be changed in place, so all names sharing them see the change.

2.3 Mutable vs immutable objects

Immutable	Cannot be changed after creation . Any "change" creates a new object .
Mutable	Can be changed in place , and the object keeps the same identity.

Point	Immutable	Mutable
Built-in types	int, float, complex, bool, str, tuple, frozenset, bytes, None	list, dict, set, bytearray
Change in place?	No	Yes
After a "change"	New object, new id	Same object, same id
Can be a dict key / set element?	Usually yes (if hashable)	No (unhashable)
Safe to share between names?	Yes	Careful: aliasing effects

```
s = "hello"
old = id(s)
s = s + " world"
print("same object?", id(s) == old)
try:
    s[0] = "H"
except TypeError as e:
    print("Error:", e)
```

Output:
same object? False
Error: 'str' object does not support item assignment

```
# mutable: list
L = [1, 2, 3]
old = id(L)
L.append(4)
print("same object?", id(L) == old, L)
```

Output:
same object? True [1, 2, 3, 4]

🕒 INTERVIEW	Tuple is immutable, but a tuple can contain a mutable object. <code>t = ([1, 2], 3)</code> then <code>t[0].append(9)</code> works, because the tuple still points to the same list; only the list's contents changed.
--------------------	--

```
t = ([1, 2], 3)
t[0].append(9)
print(t)
try:
    t[1] = 5
except TypeError as e:
    print("Error:", e)
```

Output:
`([1, 2, 9], 3)`
Error: 'tuple' object does not support item assignment

2.4 Python memory and object concept

💡 DEEP CONCEPT	This is background knowledge for interviews. You do not need it to write basic code, but it explains why the behaviour above happens.
-----------------------	--

Idea	Explanation
Everything is an object	Numbers, strings, functions, classes, even None and types themselves are objects.
Names and objects are separate	Names live in namespaces (like dictionaries mapping name → object). Objects live in a private heap .
Reference counting	Each object keeps a count of how many names/containers refer to it. When the count reaches 0, the object's memory is freed immediately.
Garbage collector	A background collector cleans reference cycles (objects that refer to each other) that reference counting alone cannot free.
<code>del x</code>	Removes the name x, not necessarily the object. The object is freed only when nothing refers to it any more.

```
import sys
a = [1, 2, 3]
print(sys.getrefcount(a))  # one extra reference is from the call itself
b = a
print(sys.getrefcount(a))
del b
print(sys.getrefcount(a))
```

Output:
2
3
2

Small integer caching and string interning

For speed, CPython **reuses** some immutable objects: small integers from -5 to 256, and many short strings that look like identifiers. So two names may point to the very same object.

```
a = 100
b = 100
print(a is b)  # small ints are cached (CPython detail)

c = 10 ** 6
d = 10 ** 6
print(c == d)  # equal values: always True
```

Output:
True
True

⚠️ COMMON MISTAKE	Never use <code>is</code> to compare numbers or strings. Whether two equal values are the same object is an implementation detail and can differ between runs, versions and interpreters. Use <code>==</code> for values. (<code>is</code> is only for None, True, False and real identity checks.)
--------------------------	--

2.5 Working with variables

Multiple assignment

```
a, b, c = 1, 2, 3          # tuple unpacking
x = y = z = 0             # chained assignment: all three share ONE object
print(a, b, c, x, y, z)
```

Output:
1 2 3 0 0 0

⚠ COMMON MISTAKE

Chained assignment with a **mutable** value: `a = b = []` makes both names refer to the **same** list. Use `a = []` and `b = []` on separate lines.

```
a = b = []
a.append(1)
print(b)
```

Output:
[1]

Swapping without a temporary variable

```
a, b = 5, 10
a, b = b, a
print(a, b)
```

Output:
10 5

⚡ SHORTCUT

`a, b = b, a` is the Pythonic swap. The right-hand side is evaluated fully first (as a tuple), then assigned.

Deleting a name

```
x = 5
del x
print(x)
```

Output:
NameError: name 'x' is not defined

Constants

Python has **no true constants**. By convention, names written in UPPER_CASE (like `MAX_SIZE = 100`) mean "please do not change this". The language will not stop you.

🎯 INTERVIEW

Python has no variable declaration and no pointers. A name is created at its first assignment. Using a name before assigning it raises `NameError`.

3. Basic Syntax

3.1 Identifiers

Definition	An identifier is a name you give to a variable, function, class, module or any other object.
Why it exists	To refer to things in your code with readable names.

Rules (these are compulsory)

- Can contain **letters** (a-z, A-Z), **digits** (0-9) and **underscore** _.
- Must **not start with a digit**.
- Are **case-sensitive**: age, Age and AGE are three different names.
- Cannot be a **keyword** (like if, class, for).
- No spaces and no special symbols (@ \$ % - . !).
- No length limit.
- (Python 3 also allows Unicode letters like é, but stick to English letters in practice.)

Identifier	Valid?	Why
student_name	Yes	letters and underscore
_count	Yes	may start with underscore
x1	Yes	digit is allowed after the first character
1x	No	starts with a digit
my-var	No	hyphen is the minus operator
my var	No	space
for	No	reserved keyword
total\$	No	\$ not allowed

Naming conventions (PEP 8)

Kind	Style	Example
Variables, functions	snake_case	total_marks, find_max()
Classes	PascalCase (CapWords)	StudentRecord
Constants	UPPER_SNAKE_CASE	MAX_LIMIT
"Private" by convention	single leading underscore	_helper
Name-mangled in classes	double leading underscore	__secret
Special / dunder methods	double underscore both sides	__init__, __str__
Throwaway variable	single underscore	for _ in range(3):

⚠ COMMON MISTAKE	Do not name variables after built-ins: <code>list = [1,2]</code> , <code>sum = 0</code> , <code>str = "a"</code> , <code>max = 5</code> , <code>input = 3</code> . This shadows the built-in, and later <code>list(...)</code> or <code>sum(...)</code> will break with a confusing error.
-------------------------	---

```
sum = 0          # shadows the built-in sum()
try:
    print(sum([1, 2, 3]))
except TypeError as e:
    print("Error:", e)
```

Output:
Error: 'int' object is not callable

🕒 INTERVIEW	Case sensitivity: <code>True</code> is a keyword, but <code>true</code> is just an undefined name. <code>print</code> is not a keyword, it is a built-in function name that you can (accidentally) overwrite.
--------------------	--

3.2 Keywords

Definition	Keywords are reserved words that have a fixed special meaning in Python. You cannot use them as identifiers.
Why it exists	They form the grammar of the language (if, for, def ...).
See the list	<code>import keyword; print(keyword.kwlist)</code>

```
import keyword
print(len(keyword.kwlist))
print(keyword.kwlist)
```

Output:

35

['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'e

Group	Keywords
Values	True False None
Logic / operators	and or not in is
Conditions	if elif else
Loops	for while break continue
Functions & classes	def return lambda yield class
Scope	global nonlocal
Exceptions	try except finally raise assert
Modules	import from as
Other	pass del with async await

★ MUST KNOW	Only True, False and None are capitalised. All other keywords are lowercase .
◆ DEEP CONCEPT	There are also soft keywords like <code>match</code> , <code>case</code> and <code>_</code> , which act as keywords only in specific places. Also, <code>print</code> and <code>exec</code> were statements in Python 2 but are ordinary functions in Python 3.

3.3 Comments

Definition	A comment is text ignored by Python. It is written for humans.
Why it exists	To explain why code does something, leave notes, or temporarily disable code.
Syntax	Start with <code>#</code> . Everything after it on that line is ignored.

```
# This is a full-line comment
x = 5  # this is an inline comment
print(x)
```

Output:

5

Multi-line comments

Python has **no special multi-line comment syntax**. Two common approaches:

```
# Line one of a long comment
# Line two of the same comment

"""
A triple-quoted string that is not assigned to anything.
Python evaluates it and throws it away.
"""
print("done")
```

Output:

done

Docstrings

```
def add(a, b):  
    """Return the sum of a and b."""  
    return a + b  
  
print(add.__doc__)
```

Output:

Return the sum of a and b.

🕒 INTERVIEW	A triple-quoted string is not really a comment . It is a string object. When it is the first statement of a function, class or module, it becomes a docstring that is stored and can be read via <code>__doc__</code> or <code>help()</code> .
⚡ SHORTCUT	In editors, select lines and press Ctrl + / (or Cmd + /) to comment or uncomment them.
⚠ COMMON MISTAKE	Do not write comments that repeat the code (<code>x = 5 # set x to 5</code>). Explain why , not what.

3.4 Basic syntax rules

Rule	Example / detail
A statement normally ends at the end of the line	No semicolon needed
Semicolons are allowed but discouraged	<code>a = 1; b = 2</code>
Python is case-sensitive	<code>Print("hi")</code> → <code>NameError</code>
Blocks start with a colon <code>:</code> and are indented	<code>if x > 0:</code>
Strings use <code>'...'</code> , <code>"..."</code> or <code>"""..."""</code>	<code>"it's"</code> and <code>'say "hi"'</code> are both fine
Long lines can be split inside <code>()</code> , <code>[]</code> , <code>{}</code>	Preferred over backslash
Backslash <code>\</code> continues a line	<code>total = 1 + 2 + \</code> (newline) <code>3</code>
Scripts start executing at the first line	No required <code>main()</code> (unlike C/C++/Java)

```
total = (1 + 2 +  
        3 + 4)      # brackets let a statement continue  
print(total)  
  
a = 1; b = 2        # legal but not recommended  
print(a + b)
```

Output:

10
3

⚠ COMMON MISTAKE	Forgetting the colon: <code>if x > 5</code> (missing <code>:</code>) gives <code>SyntaxError: expected ':'</code> . The colon is required after <code>if</code> , <code>elif</code> , <code>else</code> , <code>for</code> , <code>while</code> , <code>def</code> , <code>class</code> , <code>try</code> , <code>except</code> , <code>finally</code> , <code>with</code> .
-------------------------	---

3.5 Indentation

Definition	Indentation is the whitespace at the start of a line. In Python it is part of the syntax and defines which lines belong to a block.
Why it exists	Instead of <code>{ }</code> braces, Python uses indentation to mark blocks. This forces neat, readable code.
Standard	4 spaces per level (PEP 8).

```
x = 10
if x > 5:
    print("inside the if block")
    print("still inside")
print("outside the block")
```

Output:
inside the if block
still inside
outside the block

What happens with wrong indentation

```
x = 10
if x > 5:
print("oops")
```

Output:
IndentationError: expected an indented block after 'if' statement on line 2

```
x = 10
    y = 20
```

Output:
IndentationError: unexpected indent

Error	Cause
IndentationError: expected an indented block	Line after a colon is not indented
IndentationError: unexpected indent	A line is indented when it should not be
IndentationError: unindent does not match any outer indentation level	Dedent to a level that was never used
TabError	Tabs and spaces mixed inconsistently

★ MUST KNOW	All lines in one block must have the same indentation . Choose 4 spaces and stick with it. Configure your editor to insert 4 spaces when you press Tab.
--------------------	--

⚠ COMMON MISTAKE	Never mix tabs and spaces. They look identical on screen but Python treats them differently. This produces confusing TabErrors, especially when you paste code from the internet.
-------------------------	--

🎯 INTERVIEW	Empty block? You cannot leave it blank. Use pass (do nothing): <code>if x: pass</code> . Nested blocks add one more level of indentation each time.
--------------------	---

```
for i in range(2):
    for j in range(2):
        print(i, j)    # 2 levels deep = 8 spaces
```

Output:
0 0
0 1
1 0
1 1

4. Data Types

4.1 What is a data type?

Definition	A data type tells Python what kind of data an object holds and what operations are allowed on it.
Why it exists	You can add numbers, but not divide a string by a list. The type decides which operations make sense.
Syntax	Python decides the type automatically from the value: <code>10</code> → <code>int</code> , <code>"hi"</code> → <code>str</code> .

Type	Example	Category	Mutable?	Ordered?
int	10, -3, 0	Number	No	-
float	3.14, -0.5, 2e3	Number	No	-
complex	3+4j	Number	No	-
bool	True, False	Boolean (subclass of int)	No	-
str	"hello"	Text sequence	No	Yes
list	[1, 2, 3]	Sequence	Yes	Yes
tuple	(1, 2, 3)	Sequence	No	Yes
set	{1, 2, 3}	Collection of unique items	Yes	No
dict	{"a": 1}	Mapping (key → value)	Yes	Yes (insertion order)
NoneType	None	Absence of value	No	-

★ MUST KNOW	Learn this table by heart. Almost every question about "which data structure should I use?" is answered by mutable? ordered? duplicates? fast lookup?
--------------------	--

4.2 int (integers)

Definition	Whole numbers, positive, negative or zero, with no size limit (Python ints are arbitrary precision).
Why it exists	Counting, indexing, loops, maths.
Syntax	x = 42, y = -7, big = 1_000_000 (underscores for readability)

```
x = 10 ** 30          # would overflow in C++ / Java
print(x)
print(type(x))
print(2 ** 100)
print(1_000_000)
```

[illegible]

Number systems

Base	Prefix	Literal	Value
Binary (2)	0b	0b1010	10
Octal (8)	0o	0o12	10
Decimal (10)	none	10	10
Hexadecimal (16)	0x	0xA	10

```
print(0b1010, 0o12, 0xA)
print(bin(10), oct(10), hex(255))    # value -> string in that base
print(int("1010", 2), int("ff", 16))  # string in that base -> int
```

Output:
10 10 10
0b1010 0o12 0xff
10 255

☉ **INTERVIEW**

Python integers never overflow. They grow as memory allows. This is why big-number problems (factorials, large powers) are easy in Python. `sys.maxsize` is only the largest possible **container index**, not an integer limit.

⚠ **COMMON MISTAKE**

Leading zeros are not allowed in decimal literals: `x = 007` is a `SyntaxError`. (`00` is fine.) Also `int` has no decimals: `5/2` gives a float.

4.3 float (decimal numbers)

Definition

Numbers with a decimal point, stored as 64-bit **IEEE 754 double precision** floating-point numbers.

Why it exists

Measurements, averages, scientific calculations, ML.

Syntax

`pi = 3.14`, `x = 2.5e3 (= 2500.0)`, `y = 1e-4 (= 0.0001)`

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
print(2.5e3, 1e-4)
print(10 / 2)          # / always gives a float
```

Output:
0.30000000000000004
False
2500.0 0.0001
5.0

⚠ **COMMON MISTAKE**

Floats are approximate. Computers store them in binary, and numbers like 0.1 cannot be stored exactly. Never compare floats with `==`. Use `math.isclose(a, b)` or check `abs(a - b) < 1e-9`.

```
import math
print(math.isclose(0.1 + 0.2, 0.3))
print(abs((0.1 + 0.2) - 0.3) < 1e-9)
```

Output:
True
True

Special float values

```
inf = float("inf")
nan = float("nan")
print(inf > 10**100, -inf)
print(nan == nan)          # NaN is not equal even to itself
print(round(2.5), round(3.5), round(2.675, 2))
```

Output:
True -inf
False
2 4 2.67

☉ **INTERVIEW**

`round()` uses **banker's rounding** (round half to even): `round(2.5)` is 2, not 3. And `round(2.675, 2)` gives 2.67 because 2.675 is stored slightly below 2.675. For exact decimal maths use the `decimal` module.

◆ **DEEP CONCEPT**

A float has about **15-16 significant digits** of precision. `int + float` → float, and a very large `int` converted to float may lose precision.

4.4 complex

Definition	A number with a real part and an imaginary part: $a + bj$.
Why it exists	Engineering, signal processing, mathematics. Rare in interviews and DSA.
Syntax	$z = 3 + 4j$ or <code>complex(3, 4)</code> . Python uses <code>j</code> , not <code>i</code> .

```
z = 3 + 4j
print(z, type(z))
print(z.real, z.imag)
print(abs(z))           # magnitude = sqrt(3**2 + 4**2)
print(z.conjugate())
print(z * z)
```

Output:
(3+4j) <class 'complex'>
3.0 4.0
5.0
(3-4j)
(-7+24j)

⚠ COMMON MISTAKE	$3 + 4i$ is invalid. Also, $4j$ alone is a complex number, but j alone is just a variable name.
-------------------------	---

4.5 bool

Definition	The truth type. It has exactly two values: True and False.
Why it exists	Conditions, comparisons and decisions.
Syntax	<code>is_ready = True</code> (capital T / F)

```
print(5 > 3, 5 < 3)
print(type(True))
print(True + True)      # bool is a subclass of int
print(True * 10, False + 5)
print(isinstance(True, int))
```

Output:
True False
<class 'bool'>
2
10 5
True

★ MUST KNOW	bool is a subclass of int. <code>True == 1</code> and <code>False == 0</code> . So booleans can be added, and <code>sum([True, False, True])</code> counts the number of True values.
--------------------	--

```
marks = [80, 45, 90, 30]
passed = sum(m >= 50 for m in marks)  # counting with booleans
print(passed)
```

Output:
2

⚠ COMMON MISTAKE	<code>true</code> , <code>false</code> , <code>TRUE</code> are not valid. Write <code>True</code> and <code>False</code> exactly.
-------------------------	---

4.6 str (strings)

Definition	An immutable, ordered sequence of Unicode characters , used for text.
Why it exists	Names, messages, files, input, everything textual.
Syntax	'single', "double", """triple for multi-line""". Python has no separate char type : a character is just a string of length 1.

```
a = 'hello'
b = "it's fine"
c = """line one
line two"""
print(a, b)
print(c)
print(len(a), type(a))
```

Output:
hello it's fine
line one
line two
5 <class 'str'>

Indexing and slicing

Every character has a **position (index)**. Indexing starts from **0** on the left and from **-1** on the right.

s	=	P	Y	T	H	O	N
index		0	1	2	3	4	5
neg. index		-6	-5	-4	-3	-2	-1

```
s = "PYTHON"
print(s[0], s[-1], s[2])
print(s[1:4])      # start included, stop excluded
print(s[3:], s[3:])
print(s[:2])       # every 2nd character
print(s[::-1])     # reversed
```

Output:
P N T
YTH
PYT HON
PTO
NOHTYP

Slice syntax: `s[start:stop:step]`. **start** is included, **stop** is excluded. Defaults: start = 0, stop = end, step = 1.

Basic operations

```
print("Py" + "thon")    # concatenation
print("ab" * 3)         # repetition
print("th" in "python") # membership (substring)
print(len("python"))
s = "hello"
s = "H" + s[1:]         # "changing" = building a new string
print(s)
```

Output:
Python
ababab
True
6
Hello

★ MUST KNOW	Strings are immutable. <code>s[0] = "H"</code> raises <code>TypeError</code> . You build a new string instead.
--------------------	---

Common string methods (full detail comes in a later part)

Method	Purpose	Example → result
lower() / upper()	Change case	"Hi".upper() → "HI"
strip()	Remove spaces at both ends	" a ".strip() → "a"
split()	String → list of words	"a b c".split() → ['a', 'b', 'c']
join()	List → string	" ".join(["a", "b"]) → "a b"
replace(old, new)	Replace text	"aab".replace("a", "x") → "xxb"
find(x)	Index of first match, or -1	"hello".find("l") → 2
count(x)	How many times x appears	"banana".count("a") → 3
startswith() / endswith()	Check beginning / end	"a.py".endswith(".py") → True

Escape sequences, raw strings, f-strings

Sequence	Meaning
\n	New line
\t	Tab
\\	One backslash
\' and \"	A quote character inside a string

```
print("Line1\nLine2")
print("Path: C:\\new")
print(r"C:\new\table")      # raw string: backslashes stay as they are
name, age = "Asha", 20
print(f"{name} is {age} years old")  # f-string (formatted string)
```

Output:

```
Line1
Line2
Path: C:\new
C:\new\table
Asha is 20 years old
```

☉ INTERVIEW

Two equal strings **may or may not** be the same object. Never use `is` to compare strings. Also, `"a" < "b"` compares strings by Unicode code points (`ord()`), letter by letter.

4.7 list

Definition	An ordered, mutable collection that can hold items of any type and allows duplicates .
Why it exists	To store a changeable sequence of items: marks of students, a queue of tasks, a DSA array.
Syntax	<code>nums = [10, 20, 30]</code> , <code>empty = []</code> or <code>list()</code>

```
nums = [10, 20, 30, 20]      # duplicates allowed
mixed = [1, "two", 3.0, [4, 5]] # any types, even nested lists
print(nums[0], nums[-1], nums[1:3])
nums[0] = 99                 # mutable: change in place
nums.append(40)
print(nums, len(nums))
print(mixed[3][1])           # index into the inner list
```

Output:

```
10 20 [20, 30]
[99, 20, 30, 20, 40] 5
5
```

Common list methods

Method	What it does	Returns
<code>append(x)</code>	Add x at the end	None
<code>extend(iterable)</code>	Add each element of iterable	None
<code>insert(i, x)</code>	Insert x at index i	None
<code>remove(x)</code>	Remove first x (ValueError if absent)	None
<code>pop()</code> / <code>pop(i)</code>	Remove and return last / index i item	the item
<code>index(x)</code>	Index of first x	int
<code>count(x)</code>	Number of x	int
<code>sort()</code>	Sort in place	None
<code>reverse()</code>	Reverse in place	None
<code>copy()</code>	Shallow copy	new list
<code>clear()</code>	Remove all items	None

```
a = [1, 2, 3]
a.append([4, 5])
print(a)           # ONE new element, a list
b = [1, 2, 3]
b.extend([4, 5])
print(b)           # two new elements
print(b.append(6)) # in-place methods return None
```

Output:
`[1, 2, 3, [4, 5]]`
`[1, 2, 3, 4, 5]`
`None`

⚠ COMMON MISTAKE

`a = a.sort()` or `a = a.append(5)` makes a become None. In-place methods change the list and return None. Use `sorted(a)` if you want a new sorted list.

Time complexity (interview favourite)

Operation	Complexity	Why
<code>a[i]</code> (index)	$O(1)$	Direct position
<code>a.append(x)</code>	$O(1)$ amortized	Adds at the end
<code>a.pop()</code>	$O(1)$	Removes from the end
<code>a.insert(0, x)</code> / <code>a.pop(0)</code>	$O(n)$	All items must shift
<code>x in a</code>	$O(n)$	Scans the list
<code>a.sort()</code>	$O(n \log n)$	Timsort

⚠ COMMON MISTAKE

Nested list trap: `grid = [[0] * 3] * 3` creates 3 references to the **same** inner list. Changing one row changes all. Use `[[0] * 3 for _ in range(3)]` instead.

```
bad = [[0] * 3] * 3
bad[0][0] = 1
print(bad)
good = [[0] * 3 for _ in range(3)]
good[0][0] = 1
print(good)
```

Output:
`[[1, 0, 0], [1, 0, 0], [1, 0, 0]]`
`[[1, 0, 0], [0, 0, 0], [0, 0, 0]]`

4.8 tuple

Definition	An ordered, immutable collection. Like a list that cannot be changed after creation.
Why it exists	Safe fixed data (coordinates, RGB colours, database rows), returning multiple values from a function, and being hashable , so tuples can be dictionary keys or set elements.
Syntax	<code>point = (10, 20)</code> , <code>empty = ()</code> , <code>single = (5,)</code> (note the comma!)

```
point = (10, 20)
x, y = point                # unpacking
print(x, y, point[0])
a, *middle, b = (1, 2, 3, 4, 5) # star unpacking
print(a, middle, b)
print(type((5)), type((5,)))    # the comma makes the tuple
t = 1, 2, 3                   # brackets are optional
print(t)
```

Output:
10 20 10
1 [2, 3, 4] 5
<class 'int'> <class 'tuple'>
(1, 2, 3)

⚠ COMMON MISTAKE	(5) is just the number 5 in brackets. A one-element tuple needs a comma: (5,).
-------------------------	--

```
t = (1, 2, 3)
try:
    t[0] = 99
except TypeError as e:
    print("Error:", e)
```

Output:
Error: 'tuple' object does not support item assignment

🕒 INTERVIEW	List vs tuple: list is mutable, tuple is not. Tuples are slightly faster and use less memory, protect data from change, and can be dict keys. Use a list for changing data, a tuple for fixed records.
--------------------	---

4.9 set

Definition	An unordered collection of unique, hashable items . It is mutable.
Why it exists	Remove duplicates, super-fast membership tests (<code>x in s</code> is $O(1)$ on average), and mathematical set operations.
Syntax	<code>s = {1, 2, 3}</code> , <code>empty = set()</code> (not <code>{}</code> !)

```
s = {1, 2, 2, 3, 3, 3}
print(s)                # duplicates vanish
print(type({}), type(set())) # {} is an empty DICT
s.add(4)
s.discard(10)            # no error if missing
print(s, 3 in s)
print(list(set([5, 1, 5, 2]))) # remove duplicates from a list
```

Output:
{1, 2, 3}
<class 'dict'> <class 'set'>
{1, 2, 3, 4} True
[1, 2, 5]

Set operations

Operator	Method	Meaning	A={1,2,3}, B={3,4}
A B	union	All items in A or B	{1,2,3,4}
A & B	intersection	Items in both	{3}
A - B	difference	In A but not in B	{1,2}
A ^ B	symmetric_difference	In exactly one of them	{1,2,4}
A <= B	issubset	A inside B?	False

Method	Behaviour if item missing
remove(x)	Raises KeyError
discard(x)	Does nothing (safe)
pop()	Removes and returns an arbitrary item
clear()	Empties the set

★ MUST KNOW	Sets have no indexing (s[0] is an error) and no reliable order . Do not depend on the order you see printed. Elements must be hashable : {[1,2]} fails, {(1,2)} works.
--------------------	---

```
try:
    bad = {[1, 2], 3}
except TypeError as e:
    print("Error:", e)
print({(1, 2), 3})
```

Output:
Error: unhashable type: 'list'
{3, (1, 2)}

4.10 dict (dictionary)

Definition	A collection of key → value pairs . Keys are unique and hashable . It is mutable and (since Python 3.7) remembers insertion order .
Why it exists	Instant lookup by a meaningful key (name → marks, word → count). Average lookup is O(1).
Syntax	d = {"a": 1, "b": 2}, empty = {} or dict()

```
d = {"math": 90, "physics": 80}
print(d["math"])
d["chem"] = 70          # add
d["math"] = 95          # update
print(d.get("bio"))     # None: safe lookup
print(d.get("bio", 0))  # default value
for k, v in d.items():
    print(k, v)
print(list(d.keys()), list(d.values()))
```

Output:
90
None
0
math 95
physics 80
chem 70
['math', 'physics', 'chem'] [95, 80, 70]

Method	Meaning
d[key]	Get value, KeyError if missing
d.get(key, default)	Get value or default, never raises
d.keys(), d.values(), d.items()	Views of keys, values, (key, value) pairs
d.update(other)	Merge another dict in

Method	Meaning
d.pop(key)	Remove key and return its value
d.setdefault(key, v)	Get value; insert v first if key is missing
key in d	Membership test on keys

The frequency-counting pattern (very important)

```
arr = [1, 2, 2, 3, 3, 3]
freq = {}
for x in arr:
    freq[x] = freq.get(x, 0) + 1
print(freq)
```

Output:
{1: 1, 2: 2, 3: 3}

⚡ SHORTCUT	freq[x] = freq.get(x, 0) + 1 counts things without checking whether the key exists. collections.Counter(arr) does it in one line.
⚠ COMMON MISTAKE	d["missing"] raises KeyError. Use d.get("missing") or "missing" in d. Also, a dict key must be hashable: lists cannot be keys, tuples can.
💎 DEEP CONCEPT	Hashable means an object has a fixed hash value for its lifetime (and can be compared with ==). Immutable built-ins (int, str, tuple of hashables) are hashable; list, dict, set are not. Hashing is what makes dict and set lookups O(1).

4.11 list vs tuple vs set vs dict

Property	list	tuple	set	dict
Brackets	[]	()	{ }	{k: v}
Ordered	Yes	Yes	No	Yes (insertion order, 3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys: no, Values: yes
Indexed by position	Yes	Yes	No	No (by key)
Hashable (usable as key)	No	Yes, if items are	No	No
Membership in	O(n)	O(n)	O(1) avg	O(1) avg (keys)
Empty	[]	()	set()	{}
Best for	Changing sequence	Fixed record	Uniqueness, fast lookup	Key → value mapping

🎯 INTERVIEW	Which structure for "detect duplicates" or "Two Sum"? A set or dict. Membership in a list is O(n) but in a set/dict it is O(1) on average, which turns many O(n ²) problems into O(n).
--------------------	---

4.12 None

Definition	None is Python's special object that means " no value " or "nothing here". Its type is NoneType, and there is exactly one None object in the whole program (a singleton).
Why it exists	To represent a missing or empty result, default arguments, and functions that return nothing.

```

x = None
print(x, type(x))
print(x is None)           # correct check
print(bool(None))         # falsy
print(None == 0, None == False, None == "") # None equals nothing but None

def hello():
    print("hi")             # no return statement
r = hello()
print(r)                   # a function without return gives None
print([3, 1].sort())       # in-place methods return None

```

Output:
None <class 'NoneType'>
True
False
False False False
hi
None
None

★ MUST KNOW	Check with <code>x is None</code> (or <code>x is not None</code>), not <code>x == None</code> . Since there is only one <code>None</code> object, identity is the right and fastest test.
--------------------	--

⚠ COMMON MISTAKE	Do not confuse <code>None</code> , <code>0</code> , <code>""</code> and <code>False</code> . All are falsy, but they are different: <code>if x == 0</code> and <code>if x is None</code> mean different things. In <code>if not x</code> , you cannot tell them apart.
-------------------------	--

4.13 Type checking: `type()` and `isinstance()`

type(x)	Returns the exact type of an object.
isinstance(x, T)	Returns True if x is an instance of T or of any subclass of T . T may be a tuple of types.
Why they exist	To check what you are dealing with before using it (validation, debugging).

```

x = 42
print(type(x))
print(type(x) == int, type(x) is int)
print(isinstance(x, int))
print(isinstance(x, (int, float))) # int OR float
print(isinstance("a", (int, float)))

```

Output:
<class 'int'>
True True
True
True
False

```

print(type(True) == int) # exact type is bool
print(isinstance(True, int)) # but bool is a subclass of int
print(type([1]) is list, isinstance([1], list))

```

Output:
False
True
True True

Point	<code>type()</code>	<code>isinstance()</code>
Returns	A type object	True / False
Considers inheritance?	No, exact type only	Yes, subclasses count
Multiple types at once?	No (compare separately)	Yes: <code>isinstance(x, (int, float))</code>
True vs int	<code>type(True) == int</code> → False	<code>isinstance(True, int)</code> → True
Preferred for checks	Rarely	Usually yes

Point	<code>type()</code>	<code>isinstance()</code>
Good for	Printing / debugging the exact type	Validation, branching on type
🕒 INTERVIEW	Prefer <code>isinstance()</code> for type checks because it respects inheritance. Use <code>type(x)</code> when you need the exact type or to print it. Note: <code>isinstance(True, int)</code> is <code>True</code> , so check <code>bool</code> first if you need to treat booleans separately.	
⚠️ COMMON MISTAKE	<code>type(x) == "int"</code> is wrong: it compares a type with a string. Use <code>type(x) == int</code> or <code>isinstance(x, int)</code> .	

5. Type Conversion

5.1 What is type conversion?

Definition	Type conversion (or type casting) means changing a value from one data type to another, for example "10" → 10.
Why it exists	Data often arrives in the wrong type. <code>input()</code> always gives text, files give text, but you need numbers to calculate.

Point	Implicit conversion	Explicit conversion
Also called	Type coercion / promotion	Type casting
Who does it?	Python, automatically	You, by calling <code>int()</code> , <code>float()</code> , <code>str()</code> ...
When?	Mixed numeric operations	Whenever you write the function call
Example	<code>3 + 4.5 → 7.5</code>	<code>int("25") → 25</code>
Risk	Very limited (only safe numeric widening)	Can raise <code>ValueError</code> / <code>TypeError</code>
Data loss possible?	No (widening only)	Yes: <code>int(3.9) → 3</code>

5.2 Implicit conversion

Python automatically converts to the **wider** numeric type to avoid losing information. The order is:

```
bool -> int -> float -> complex
(narrow)                (wide)
```

```
print(3 + 4.5, type(3 + 4.5))      # int + float -> float
print(True + 5, type(True + 5))    # bool + int -> int
print(2 + 3j)                      # float/int + complex -> complex
print(10 / 2, type(10 / 2))        # true division always gives float
print(1 == 1.0, 1 == True)        # compared after conversion
```

Output:
7.5 <class 'float'>
6 <class 'int'>
(2+3j)
5.0 <class 'float'>
True True

★ MUST KNOW	Python never implicitly converts between text and numbers. <code>"5" + 5</code> is an error. Only numeric types (<code>bool</code> , <code>int</code> , <code>float</code> , <code>complex</code>) mix automatically.
--------------------	--

```
print("Age: " + 20)
```

Output:
`TypeError: can only concatenate str (not "int") to str`

5.3 Explicit conversion

Function	Converts to	Examples
<code>int(x)</code>	integer	<code>int("10") → 10</code> , <code>int(3.9) → 3</code> , <code>int("1010", 2) → 10</code>
<code>float(x)</code>	float	<code>float("3.14") → 3.14</code> , <code>float(5) → 5.0</code>
<code>str(x)</code>	string	<code>str(100) → "100"</code> , <code>str([1,2]) → "[1, 2]"</code>
<code>bool(x)</code>	boolean	<code>bool(0) → False</code> , <code>bool("a") → True</code>
<code>list(x)</code>	list	<code>list("abc") → ['a', 'b', 'c']</code>
<code>tuple(x)</code>	tuple	<code>tuple([1,2]) → (1, 2)</code>
<code>set(x)</code>	set	<code>set([1,1,2]) → {1, 2}</code>
<code>dict(x)</code>	dict	<code>dict([("a",1), ("b",2)]) → {'a': 1, 'b': 2}</code>
<code>complex(a, b)</code>	complex	<code>complex(3, 4) → (3+4j)</code>
<code>chr(n) / ord(c)</code>	char ↔ Unicode code	<code>ord("A") → 65</code> , <code>chr(97) → 'a'</code>

What works and what fails (real results)

Expression	Result / error
<code>int("10")</code>	10
<code>int(" 42 ")</code>	42
<code>int("3.5")</code>	ValueError
<code>int(3.9)</code>	3
<code>int(-3.9)</code>	-3
<code>int("abc")</code>	ValueError
<code>int("1_000")</code>	1000
<code>int("ff", 16)</code>	255
<code>int(None)</code>	TypeError
<code>float("3.14")</code>	3.14
<code>float("1e3")</code>	1000.0
<code>float("abc")</code>	ValueError
<code>float(5)</code>	5.0
<code>str(3.0)</code>	'3.0'
<code>str(None)</code>	'None'
<code>str([1, 2])</code>	'[1, 2]'
<code>bool("")</code>	False
<code>bool("False")</code>	True
<code>bool(0.0)</code>	False
<code>bool([0])</code>	True
<code>list("abc")</code>	['a', 'b', 'c']
<code>list(5)</code>	TypeError
<code>list({"a": 1, "b": 2})</code>	['a', 'b']
<code>tuple("hi")</code>	('h', 'i')
<code>set([1, 1, 2])</code>	{1, 2}
<code>set("hello") == set("olleh")</code>	True
<code>dict([("a", 1), ("b", 2)])</code>	{'a': 1, 'b': 2}
<code>dict([1, 2])</code>	TypeError
<code>ord("a")</code>	97
<code>chr(65)</code>	'A'

☉ INTERVIEW	<code>int()</code> truncates toward zero (<code>int(-3.9) → -3</code>), while <code>round()</code> rounds to nearest (ties to even) and <code>math.floor()</code> rounds down (<code>floor(-3.9) → -4</code>).
⚠ COMMON MISTAKE	<code>int("3.5")</code> fails even though 3.5 looks like a number. Convert in two steps: <code>int(float("3.5"))</code> .
⚠ COMMON MISTAKE	<code>bool("False")</code> is True. Any non-empty string is truthy , even "False" or "0".
♦ DEEP CONCEPT	<code>str(x)</code> gives a human-friendly text, while <code>repr(x)</code> gives an unambiguous developer text: <code>str("hi") → hi</code> , <code>repr("hi") → 'hi'</code> . The REPL shows <code>repr</code> , print shows <code>str</code> .

```
n = "25"
print(n * 2)          # string repetition, not arithmetic
print(int(n) * 2)     # after conversion
print(str(25) + "5")
print(list("abc"), list(range(4)))
```

Output:

```
2525
50
255
['a', 'b', 'c'] [0, 1, 2, 3]
```

6. Input and Output

6.1 Input with input()

Definition	<code>input()</code> pauses the program, waits for the user to type a line and press Enter, and returns it as a string .
Why it exists	To let programs receive data from the user or from test cases.
Syntax	<code>variable = input("optional prompt")</code>

```
name = input("Enter name: ")
print("Hello,", name)
```

Keyboard input:

Riya

Output:

Enter name: Hello, Riya

★ MUST KNOW

`input()` **ALWAYS** returns a `str`, even if the user types digits. You must convert it: `int(input())`, `float(input())`.

```
a = input()
b = input()
print(a + b)           # string concatenation
print(int(a) + int(b)) # numeric addition
```

Keyboard input:

5

7

Output:

57

12

Standard input patterns (learn these until they are automatic)

Goal	Code	Sample input → result
One integer	<code>n = int(input())</code>	5 → 5
One float	<code>x = float(input())</code>	2.5 → 2.5
One string	<code>s = input()</code>	hello → "hello"
Two integers, one line	<code>a, b = map(int, input().split())</code>	3 4 → a=3, b=4
List of integers	<code>arr = list(map(int, input().split()))</code>	1 2 3 → [1, 2, 3]
List of words	<code>words = input().split()</code>	a b c → ['a', 'b', 'c']
n lines of ints	<code>rows = [int(input()) for _ in range(n)]</code>	n lines

```
a, b = map(int, input().split())
print(a + b)
arr = list(map(int, input().split()))
print(arr, sum(arr))
```

Keyboard input:

3 4

10 20 30

Output:

7

[10, 20, 30] 60

How `map(int, input().split())` works, step by step: `input()` returns "3 4" → `.split()` gives ["3", "4"] → `map(int, ...)` applies `int` to each piece → the two values are unpacked into `a` and `b`.

⚠ COMMON MISTAKE

Forgetting to convert: `n = input()` then `range(n)` or `n + 1` causes a `TypeError`. Also `a, b = input().split()` gives **strings**, not integers.

⚠ COMMON MISTAKE	<code>a, b = map(int, input().split())</code> crashes with <code>ValueError</code> if the user does not type exactly two numbers.
✂ SHORTCUT	For large inputs in competitive programming, <code>import sys; input = sys.stdin.readline</code> is much faster (note: it keeps the trailing newline, but <code>int()</code> and <code>split()</code> ignore it).

6.2 Output with `print()`

Definition	<code>print()</code> displays values on the screen (standard output). It converts each value to a string with <code>str()</code> and adds a newline at the end.
Syntax	<code>print(*values, sep=" ", end="\n", file=sys.stdout, flush=False)</code>

```
print("Hello")
print("a", "b", "c")           # separated by a space
print("a", "b", "c", sep="-")  # custom separator
print("Loading", end="...")    # no newline at the end
print("done")
print()                        # blank line
print(10, 3.5, True, None, [1, 2]) # any types
```

Output:

```
Hello
a b c
a-b-c
Loading...done
```

```
10 3.5 True None [1, 2]
```

Parameter	Default	Meaning
<code>sep</code>	<code>" "</code>	Text placed between the values
<code>end</code>	<code>"\n"</code>	Text placed after the last value
<code>file</code>	<code>sys.stdout</code>	Where to send the output (e.g. a file)
<code>flush</code>	<code>False</code>	Force the text to appear immediately

Formatting output

```
name, marks = "Asha", 87.456
print("Name:", name, "Marks:", marks)           # commas add spaces
print("Name: " + name)                          # + needs strings only
print(f"{name} scored {marks:.2f}")              # f-string, 2 decimals
print("{} scored {:.1f}".format(name, marks))    # str.format()
print("%s scored %d" % (name, 87))              # old % style
n = 255
print(f"{n:5d}|{n:<5d}|{n:05d}|{n:,}|{n:b}|{n:x}")
x = 7
print(f"x={x}")                                # debug form (3.8+)
```

Output:

```
Name: Asha Marks: 87.456
Name: Asha
Asha scored 87.46
Asha scored 87.5
Asha scored 87
 255|255 |00255|255|1111111|ff
x=7
```

Format spec	Meaning	Example (<code>x = 3.14159 / n = 42</code>)
<code>:.2f</code>	2 digits after the decimal point	3.14
<code>:5d</code>	Width 5, right-aligned	42
<code>:<5 / :>5 / :^5</code>	Left / right / centre in width 5	42
<code>:05d</code>	Pad with zeros	00042
<code>:,</code>	Thousands separator	1,000,000

Format spec	Meaning	Example (x = 3.14159 / n = 42)
:b / :x / :o	Binary / hex / octal	101010 / 2a / 52
:.1%	Percentage	0.256 → 25.6%

Printing collections

```
arr = [1, 2, 3]
print(arr)           # [1, 2, 3]
print(*arr)          # 1 2 3    (star unpacks the list)
print(*arr, sep="\n") # each on its own line
print(" ".join(map(str, arr))) # join needs strings
```

Output:

```
[1, 2, 3]
1 2 3
1
2
3
1 2 3
```

⚡ SHORTCUT	print(*arr) prints list elements separated by spaces, the usual output format in competitive programming.
⚠ COMMON MISTAKE	print("Total: " + 5) → TypeError (cannot join str and int). Use a comma print("Total:", 5) or an f-string print(f"Total: {5}").
🕒 INTERVIEW	print() itself returns None. x = print("hi") prints hi and stores None. This is why print inside a function is not the same as return: printing shows a value, returning hands it back to the caller.

```
def add_print(a, b):
    print(a + b)
def add_return(a, b):
    return a + b
r1 = add_print(2, 3)
r2 = add_return(2, 3)
print(r1, r2)
```

Output:

```
5
None 5
```

7. Operators

7.1 What are operators?

Definition	An operator is a symbol that performs an operation. The values it works on are called operands . In <code>5 + 3</code> , <code>+</code> is the operator and <code>5</code> , <code>3</code> are operands.
Why it exists	To calculate, compare, combine conditions, assign values and work with bits.

Group	Operators	Result type
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>//</code> <code>%</code> <code>**</code>	number
Comparison	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code>	bool
Logical	<code>and</code> <code>or</code> <code>not</code>	one of the operands
Assignment	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>//=</code> <code>%=</code> <code>**=</code> <code>&=</code> <code> =</code> <code>^=</code> <code>>>=</code> <code><<=</code> <code>:=</code>	-
Membership	<code>in</code> <code>not in</code>	bool
Identity	<code>is</code> <code>is not</code>	bool
Bitwise	<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code>	int

Operators are also grouped by **how many operands** they need: **unary** (one: `-x`, `not x`, `~x`), **binary** (two: `a + b`), and **ternary** (three: `a if cond else b`).

7.2 Arithmetic operators

Operator	Name	Example	Result
<code>+</code>	Addition	<code>7 + 2</code>	9
<code>-</code>	Subtraction	<code>7 - 2</code>	5
<code>*</code>	Multiplication	<code>7 * 2</code>	14
<code>/</code>	True division (always float)	<code>7 / 2</code>	3.5
<code>//</code>	Floor division	<code>7 // 2</code>	3
<code>%</code>	Modulus (remainder)	<code>7 % 2</code>	1
<code>**</code>	Exponent (power)	<code>7 ** 2</code>	49

```
print(7 + 2, 7 - 2, 7 * 2)
print(7 / 2, 8 / 2)        # / always gives float
print(7 // 2, 7 % 2, 7 ** 2)
print(divmod(17, 5))       # (quotient, remainder) together
print(2 ** 10, 2 ** -1, 9 ** 0.5)
```

Output:
9 5 14
3.5 4.0
3 1 49
(3, 2)
1024 0.5 3.0

Floor division and modulus with negative numbers

`//` **rounds down toward negative infinity** (it is *floor*, not truncation). `%` always takes the **sign of the divisor**. The two are linked by the rule: `a == (a // b) * b + a % b`.

```
print(-7 // 2, 7 // -2)    # -4 and -4 (not -3)
print(-7 % 3, 7 % -3)     # 2 and -2
print(7.5 // 2, 7.5 % 2)  # works on floats too
a, b = -7, 3
print(a == (a // b) * b + a % b)
```

Output:

```
-4 -4
2 -2
3.0 1.5
True
```

🕒 INTERVIEW	-7 // 2 is -4, not -3. In C/C++/Java the result is -3 (truncation). This difference is a classic interview and competitive-programming trap. Also, Python's % on negatives always gives a non-negative result for a positive divisor, which is handy for circular indexing.
--------------------	---

⚠️ COMMON MISTAKE	^ is not power in Python, it is bitwise XOR. Use ** for power: 2 ** 3 is 8, 2 ^ 3 is 1.
--------------------------	--

⚠️ COMMON MISTAKE	Dividing by zero raises ZeroDivisionError for /, // and %.
--------------------------	--

```
print(2 ** 3 ** 2)    # ** is right-associative: 2 ** (3 ** 2)
print(-2 ** 2)        # ** binds tighter than unary minus
print((-2) ** 2)
print(0 ** 0)
print(10 - 4 - 3)     # left to right: (10 - 4) - 3
```

Output:

```
512
-4
4
1
3
```

Arithmetic operators on non-numbers

```
print("ab" + "cd", "ab" * 3)
print([1, 2] + [3], [0] * 4)
print((1, 2) + (3,))
```

Output:

```
abcd ababab
[1, 2, 3] [0, 0, 0, 0]
(1, 2, 3)
```

⚡ SHORTCUT	Handy modulo tricks: n % 2 == 0 even; n % 10 last digit; n // 10 removes last digit; arr[(i + 1) % n] circular next index; n % k == 0 divisible by k.
-------------------	--

7.3 Comparison (relational) operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater than or equal	5 >= 5	True
<=	Less than or equal	3 <= 2	False

```
print(1 == 1.0, 1 == True, "a" == "a")
print("apple" < "banana")           # compared letter by letter
print([1, 2, 3] < [1, 2, 4])         # lists compare element by element
print((1, 2) == (1, 2), [1, 2] == (1, 2))
```

Output:

```
True True True
True
True
True False
```

Chained comparisons (a Python special)

```
x = 5
print(1 < x < 10)           # means (1 < x) and (x < 10)
print(1 < x < 3)
print(1 < 2 < 3 < 4)
print(1 == 1 < 2)
```

Output:

```
True
False
True
True
```

⚡ **SHORTCUT**

Write $0 \leq i < n$ instead of $i \geq 0$ and $i < n$. It reads like mathematics and each part is evaluated only once.

⚠ **COMMON MISTAKE**

Do not confuse `=` (assignment) with `==` (comparison). if `x = 5:` is a `SyntaxError`.

```
print(1 < "a")
```

Output:

```
TypeError: '<' not supported between instances of 'int' and 'str'
```

🔗 **INTERVIEW**

Comparing incompatible types with `<`, `>` raises `TypeError` in Python 3 (`1 < "a"`). But `==` never raises: `1 == "a"` is simply `False`.

7.4 Logical operators

Operator	Meaning	Truth rule
and	Both must be true	True only if both are true
or	At least one true	False only if both are false
not	Reverses	not True → False

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

```
age, has_id = 20, True
print(age >= 18 and has_id)
print(age < 18 or has_id)
print(not has_id)
```

Output:

```
True
True
False
```

Short-circuit evaluation and returned values

and and or **stop as soon as the answer is known**, and they **return one of the operands**, not necessarily True/False.

Expression	Rule	Returns
a and b	If a is falsy return a, else return b	first falsy value, or the last value
a or b	If a is truthy return a, else return b	first truthy value, or the last value
not a	Always a bool	True / False

```
print(3 and 4, 0 and 4, "" and "x")
print(3 or 4, 0 or 4, "" or "default")
print(None or [] or "last")
print(not 0, not "hello", not [])
x = 0
print(x != 0 and 10 / x > 1)    # safe: right side never runs
```

Output:

```
4 0
3 4 default
last
True False True
False
```

⚡ SHORTCUT	name = user_input or "Guest" gives a default when the input is empty. Short-circuit also protects you: if arr and arr[0] == 5: never fails on an empty list.
⚠ COMMON MISTAKE	if x == 1 or 2: is always true . It means (x == 1) or 2, and 2 is truthy. Write if x == 1 or x == 2: or if x in (1, 2):.
🎯 INTERVIEW	Precedence among logical operators: not is higher than and, which is higher than or. So True or False and False is True or (False and False) → True.

7.5 Assignment operators

Operator	Example	Same as
=	x = 5	assign
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
//=	x //= 3	x = x // 3
%=	x %= 3	x = x % 3
**=	x **= 3	x = x ** 3
&= = ^=	x &= 3	x = x & 3 (same for , ^)
>>= <<=	x >>= 1	x = x >> 1
:= (walrus, 3.8+)	if (n := len(a)) > 3:	assign inside an expression

```
x = 10
x += 5; print(x)
x /= 2; print(x)
x **= 2; print(x)
a = b = 3          # chained
a, b = 1, 2        # multiple
a, b = b, a        # swap
print(a, b)
data = [1, 2, 3, 4]
if (n := len(data)) > 3:
    print("long list with", n, "items")
```

Output:

```
15
7
49
2 1
long list with 4 items
```

⚠ COMMON MISTAKE

Python has **no ++ or --**. `x++` is a `SyntaxError`. Use `x += 1`. Even trickier, `++x` is legal but does nothing (it is `+(+x)`), so while `++i < 5` silently misbehaves.

```
x = 5
++x
print(x)
```

Output:

```
5
```

🕒 INTERVIEW

`+=` **on a list modifies it in place, but** `a = a + [...]` **creates a new list**. This matters when several names share one list.

```
a = [1]; b = a
a += [2]          # in place: b sees it
print(b)
a = [1]; b = a
a = a + [2]       # new list: b is unchanged
print(b)
n = 5; m = n
n += 1            # ints are immutable: m unchanged
print(m)
```

Output:

```
[1, 2]
[1]
5
```

7.6 Membership operators

Definition

in checks whether a value exists inside a container. not in checks that it does not. Both return a bool.

Works on

str (substring), list, tuple, set, dict (**keys only**), range.

```
print(3 in [1, 2, 3], 10 not in [1, 2, 3])
print("py" in "python", "z" in "python")
d = {"a": 1, "b": 2}
print("a" in d, 1 in d)          # dict checks KEYS, not values
print(1 in d.values())
print(2 in range(5), (1, 2) in {(1, 2), (3, 4)})
```

Output:

```
True True
True False
True False
True
True True
```

Container	Cost of <code>x in c</code>	Notes
list, tuple, str	$O(n)$	Scans item by item
set	$O(1)$ average	Uses hashing
dict (keys)	$O(1)$ average	Uses hashing
range	$O(1)$	Computed arithmetically

⚠ COMMON MISTAKE	<code>1 in "123"</code> raises <code>TypeError</code> (substring test needs a string). Also <code>value in dict</code> tests keys, not values.
🔗 INTERVIEW	If you test membership repeatedly, convert the list to a set first: <code>s = set(arr)</code> . This is the core of the Two-Sum and "contains duplicate" tricks.

7.7 Identity operators

Definition	is checks whether two names refer to the same object (same id). <code>is not</code> is the opposite.
Why it exists	Sometimes you care about the object itself, not its value. The main legitimate use is comparing to <code>None</code> .

```

a = [1, 2]
b = [1, 2]
c = a
print(a == b, a is b)    # equal values, different objects
print(a is c)            # same object
print(id(a) == id(c))    # is <=> same id
x = None
print(x is None, x is not None)

```

Output:
True False
True
True
True False

7.8 `==` vs `is` (very important)

Point	<code>==</code>	<code>is</code>
Checks	Value equality	Object identity (same object in memory)
Equivalent to	<code>a.__eq__(b)</code> (can be customised)	<code>id(a) == id(b)</code>
Can it be overridden?	Yes, by a class	No
Use for	Comparing numbers, strings, lists...	<code>None</code> , <code>True</code> , <code>False</code> , checking aliasing
Speed	Depends on the data	Always $O(1)$
<code>[1,2] ? [1,2]</code>	True	False

```

print([] == [], [] is [])
s1 = [1, 2, 3]
s2 = s1.copy()
print(s1 == s2, s1 is s2)    # copy: equal but distinct
print(s1 is s1[:], s1 == s1[:])
print(1 == 1.0, 1 is 1.0)

```

Output:
True False
True False
False True
True False

★ MUST KNOW	Use <code>==</code> to compare values. Use <code>is</code> only for <code>None</code> (and rarely <code>True/False</code>). if <code>x is None</code> : is correct; if <code>x == None</code> : works but is poor style; if <code>x is 5</code> : is a bug.
--------------------	---

⚠ COMMON MISTAKE	Small ints and short strings may look like is works: <code>a = 5; b = 5; a is b</code> gives True in CPython only because of caching . <code>a = 1000; b = 1000</code> may give False in another context. Python 3.8+ even prints a SyntaxWarning for <code>x is 5</code> .
-------------------------	--

```
a = 256; b = 256
print(a is b)          # True in CPython: small ints are cached (-5..256)
x = int("1000"); y = int("1000")
print(x == y, x is y)  # equal, but two separate objects
```

Output:
True
True False

🕒 INTERVIEW	Classic question: "What is the difference between == and is?" Answer: == compares values , is compares identity . Two lists with the same contents are == but not is. Then mention that is should be used for None checks and that integer/string caching is an implementation detail.
--------------------	---

7.9 Bitwise operators

Definition	Bitwise operators work on the binary digits (bits) of integers.
Why it exists	Fast low-level tricks: check odd/even, power of two, swapping, flags, subset masks, and many competitive-programming and interview problems.

Operator	Name	Rule (per bit)	Example (a=5=101, b=3=011)
&	AND	1 only if both bits are 1	<code>5 & 3 = 001 = 1</code>
	OR	1 if at least one bit is 1	<code>5 3 = 111 = 7</code>
^	XOR	1 if the bits are different	<code>5 ^ 3 = 110 = 6</code>
~	NOT (invert)	Flips all bits	<code>~5 = -6</code>
<<	Left shift	Shift bits left, fill with 0	<code>5 << 1 = 1010 = 10</code>
>>	Right shift	Shift bits right	<code>5 >> 1 = 10 = 2</code>

```
a, b = 5, 3
print(bin(a), bin(b))
print(a & b, a | b, a ^ b)
print(~a)                # -(a + 1)
print(a << 1, a << 3)     # multiply by 2**k
print(a >> 1, -5 >> 1)    # floor divide by 2**k
print(bin(a & b), bin(a | b), bin(a ^ b))
```

Output:
0b101 0b11
1 7 6
-6
10 40
2 -3
0b1 0b111 0b110

★ MUST KNOW	<code>x << k</code> equals <code>x * 2**k</code> and <code>x >> k</code> equals <code>x // 2**k</code> . <code>~x</code> equals <code>-x - 1</code> (Python ints behave as if they have infinitely many sign bits, using two's complement).
--------------------	---

Bit tricks used in interviews

Task	Code	Explanation
Is n odd?	<code>n & 1</code>	Last bit is 1 for odd numbers
Multiply / divide by 2	<code>n << 1, n >> 1</code>	Shift by one position
Get bit k (0 = rightmost)	<code>(n >> k) & 1</code>	Shift the bit to the end, mask it
Set bit k	<code>n (1 << k)</code>	OR with a mask
Clear bit k	<code>n & ~(1 << k)</code>	AND with an inverted mask
Toggle bit k	<code>n ^ (1 << k)</code>	XOR flips one bit
Power of two?	<code>n > 0 and n & (n - 1) == 0</code>	Powers of 2 have exactly one 1-bit
Remove lowest set bit	<code>n & (n - 1)</code>	Used to count set bits
Count set bits	<code>bin(n).count("1")</code> or <code>n.bit_count()</code>	<code>bit_count()</code> needs Python 3.10+
Swap without temp	<code>a ^= b; b ^= a; a ^= b</code>	XOR trick (a, b = b, a is simpler)
Find the unique number	XOR of all items	$x \oplus x = 0$ and $x \oplus 0 = x$

```
n = 12          # 1100
print(n & 1, (n >> 2) & 1)
print(n | (1 << 0), n & ~(1 << 2), n ^ (1 << 1))
print(16 & 15 == 0, 18 & 17 == 0)      # power of two check
print(bin(n).count("1"), n.bit_count())
print(2 ^ 2 ^ 3 ^ 3 ^ 7)               # duplicates cancel: unique is 7
```

Output:
0 1
13 8 14
True False
2 2
7

🕒 INTERVIEW	Single Number problem: in a list where every element appears twice except one, XOR of all elements gives the unique one, in $O(n)$ time and $O(1)$ space.
⚠️ COMMON MISTAKE	In Python, <code>a & b == c</code> is parsed as <code>(a & b) == c</code> because bitwise operators bind tighter than comparisons (the opposite of C/C++). Still, use brackets for readability. Also, shifting by a negative count raises <code>ValueError</code> .

7.10 Operator precedence

Definition	Precedence decides which operator is evaluated first when an expression has several. Associativity decides the order for operators of the same level (mostly left to right).
Rule of thumb	When unsure, use parentheses . They make code clearer and always win.

Level (high → low)	Operators	Associativity
1	<code>()</code> parentheses	-
2	<code>**</code>	right to left
3	unary <code>+x</code> <code>-x</code> <code>~x</code>	right to left
4	<code>*</code> <code>/</code> <code>//</code> <code>%</code>	left to right
5	<code>+</code> <code>-</code>	left to right
6	<code><<</code> <code>>></code>	left to right
7	<code>&</code>	left to right
8	<code>^</code>	left to right
9	<code> </code>	left to right
10	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code> <code>is</code> <code>is not</code> <code>in</code> <code>not in</code> (all same level, chainable)	chained

Level (high → low)	Operators	Associativity
11	not	right to left
12	and	left to right
13	or	left to right
14	a if c else b (ternary), lambda	-
15	:= (walrus)	-

```

print(2 + 3 * 4)          # 14, not 20
print((2 + 3) * 4)        # 20
print(2 ** 3 ** 2)        # 512 (right to left)
print(-3 ** 2, (-3) ** 2)
print(100 / 10 * 2)       # left to right: (100/10)*2
print(10 - 2 ** 3 // 2)   # 10 - ((2**3) // 2)
print(1 + 2 << 3)         # + before <<: (1+2) << 3
print(True or False and False)
print(not True == False)  # not (True == False)
print(5 > 3 and 2 < 1 or 4 > 1)
print(1 | 2 & 3 ^ 4)

```

Output:

```

14
20
512
-9 9
20.0
6
24
True
True
True
7

```

★ MUST KNOW	Order to remember: () → ** → unary → * / // % → + - → shifts → & → ^ → → comparisons /in/is → not → and → or.
⚠ COMMON MISTAKE	-2 ** 2 is -4, not 4, because ** is stronger than the unary minus. And not x == y means not (x == y).
🕒 INTERVIEW	Precedence never changes evaluation order of operands left to right in a function call or expression, only how they are grouped.

Ternary (conditional) expression

```

x = 7
result = "positive" if x > 0 else "non-positive"
print(result)
print("even" if x % 2 == 0 else "odd")

```

Output:

```

positive
odd

```

Syntax: value_if_true if condition else value_if_false. It is an expression, so it can be used anywhere a value is expected.

8. Truthy and Falsy Values

Definition	In a condition (if, while, and, or, not), Python treats every object as either "true-like" (truthy) or "false-like" (falsy). It does not have to be the actual True or False.
Why it exists	Makes code shorter and more readable: <code>if items:</code> instead of <code>if len(items) > 0:</code> .
Check with	<code>bool(value)</code>

The falsy values (memorise these)

Falsy value	Type
False	bool
None	NoneType
0, 0.0, 0j	zero of any numeric type
""	empty string
[] () {} set() range(0)	empty containers

Everything else is truthy: non-zero numbers (including negatives), non-empty strings and containers, functions, most objects.

```
values = [0, 1, -1, 0.0, "", " ", "0", "False", [], [0], {}, {0: 0}, None, set(), (), (0,)]
for v in values:
    print(repr(v).ljust(8), bool(v))
```

Output:

```
0      False
1      True
-1     True
0.0    False
''     False
' '    True
'0'    True
'False' True
[]     False
[0]    True
{}     False
{0: 0} True
None   False
set()  False
()     False
(0,)   True
```

```
items = []
if items:
    print("has items")
else:
    print("empty")

name = "Asha"
if name:
    print("Hello", name)      # non-empty string is truthy
```

Output:

```
empty
Hello Asha
```

< SHORTCUT	Prefer <code>if arr:</code> over <code>if len(arr) > 0:</code> and <code>if not arr:</code> over <code>if len(arr) == 0:</code> . It is the Pythonic style and works for lists, strings, dicts and sets.
⚠ COMMON MISTAKE	<code>bool("False")</code> and <code>bool("0")</code> are True , because the strings are non-empty. Also <code>[0]</code> is truthy: it is a non-empty list, whatever it contains.
⚠ COMMON MISTAKE	Use <code>is None</code> when <code>0</code> or <code>""</code> are valid values. <code>if not x:</code> cannot tell <code>None</code> , <code>0</code> , <code>""</code> and <code>[]</code> apart.

```
def describe(score):
    if not score:
        return "no score"
    return "score " + str(score)

def describe_ok(score):
    if score is None:
        return "no score"
    return "score " + str(score)

print(describe(0), "|", describe_ok(0))
```

Output:

no score | score 0

◆ **DEEP CONCEPT**

Custom classes are truthy by default. To change that, define `__bool__()` (or `__len__()`, where length 0 means falsy). That is how empty lists know they are falsy.

◎ **INTERVIEW**

and / or return operands, not booleans: `[]` or `"x"` → `"x"`, `5` and `0` → `0`. `not` always returns a real bool. `bool(float("nan"))` is `True`.

9. Common Beginner Mistakes

Most bugs in the first months come from a short list of causes. Read this list once now and again after your first few programs.

#	Mistake	Wrong	Correct / why
1	Missing colon	<code>if x > 5</code>	<code>if x > 5:</code>
2	Bad / mixed indentation	body not indented, tabs + spaces	4 spaces everywhere
3	= instead of ==	<code>if x = 5:</code>	<code>if x == 5:</code>
4	Forgetting <code>input()</code> returns str	<code>n = input(); n + 1</code>	<code>n = int(input())</code>
5	Joining str and int	<code>"Age: " + 20</code>	<code>"Age:", 20</code> or <code>f"Age: {20}"</code>
6	Using ^ for power	<code>2 ^ 3</code> (gives 1)	<code>2 ** 3</code>
7	++ / --	<code>x++</code>	<code>x += 1</code>
8	Comparing floats with ==	<code>0.1 + 0.2 == 0.3</code>	<code>math.isclose(...)</code>
9	Using <code>is</code> for values	<code>if x is 5:</code>	<code>if x == 5:</code>
10	<code>x == None</code>	<code>if x == None:</code>	<code>if x is None:</code>
11	Assuming <code>b = a</code> copies	<code>b = a</code> (lists)	<code>b = a.copy()</code>
12	Assigning result of in-place method	<code>a = a.sort()</code>	<code>a.sort()</code> or <code>a = sorted(a)</code>
13	Empty set as {}	<code>s = {}</code> (a dict!)	<code>s = set()</code>
14	Single-element tuple	<code>t = (5)</code> (an int)	<code>t = (5,)</code>
15	<code>[[0]*3]*3</code> grid	rows share one list	<code>[[0]*3 for _ in range(3)]</code>
16	Shadowing built-ins	<code>list = [1], sum = 0</code>	Use <code>lst</code> , <code>total</code>
17	Using <code>or</code> with two values	<code>if x == 1 or 2:</code>	<code>if x in (1, 2):</code>
18	Off-by-one with range	<code>range(1, n)</code> misses <code>n</code>	<code>range(1, n + 1)</code>
19	Wrong capitalisation	<code>true, none, Print()</code>	<code>True, None, print()</code>
20	Expecting / to return int	<code>7 / 2 = 3.5</code>	use <code>//</code> for integer result

Reading error messages

Error	Typical cause	Example
<code>SyntaxError</code>	Invalid Python grammar	missing <code>:</code> , bracket or quote
<code>IndentationError</code> / <code>TabError</code>	Wrong or mixed indentation	body not indented
<code>NameError</code>	Name used before it exists (or typo)	<code>print(totl)</code>
<code>TypeError</code>	Wrong type for an operation	<code>"a" + 1, len(5)</code>
<code>ValueError</code>	Right type, unacceptable value	<code>int("abc")</code>
<code>ZeroDivisionError</code>	Divide or modulo by zero	<code>5 / 0</code>
<code>IndexError</code>	List index out of range	<code>[1,2][5]</code>
<code>KeyError</code>	Missing dict key	<code>{1:"a"}</code>
<code>AttributeError</code>	Object has no such attribute / method	<code>"abc".push(1)</code>

✂ SHORTCUT

Read the **last line** of a traceback first: it names the error and gives the reason. The line above it points to the exact code line.

10. Part 1 Quick Revision

Read this the day before an exam or interview. Each block is the shortest useful summary of a chapter.

Meet Python

- Python = high-level, interpreted (compiled to **bytecode**, run by the **PVM**), dynamically **and strongly** typed, object-oriented, huge library. Created by Guido van Rossum, 1991.
- Compiler: whole program → machine code first (fast run). Interpreter: step by step (easy debug, portable).
- Weak points: slower than C/C++, GIL limits CPU-bound threads, more memory.
- Dynamic typing: **type lives in the object, not the name**. `x = 1`; `x = "a"` is fine. Type hints are not enforced.

Variables, objects and memory

- **Name → object**. Assignment binds a name to an object; it never copies. `b = a` makes an alias.
- Every object: **identity** (id), **type** (type), **value**.
- Immutable: int, float, complex, bool, str, tuple, frozenset, None. Mutable: list, dict, set.
- Reference counting frees objects; a garbage collector handles cycles. `del x` removes only the name.
- Small ints (-5..256) and some strings are cached: `is` on values is unreliable.
- `a, b = b, a` swaps. `a = b = []` shares one list.

Basic syntax

- Identifier: letters, digits, `_`; not start with digit; case-sensitive; not a keyword. Style: snake_case, PascalCase, UPPER_CASE.
- 35 keywords (3.12); only True, False, None capitalised. `print` is not a keyword.
- Comments start with `#`; triple-quoted strings are not comments, but docstrings.
- Blocks: colon + **4 spaces**. Do not mix tabs and spaces. Empty block → `pass`.

Data types

Type	Key facts
int	Unlimited size. <code>0b</code> , <code>0o</code> , <code>0x</code> prefixes. <code>_</code> allowed in literals.
float	64-bit, approximate. <code>0.1+0.2 != 0.3</code> . Use <code>math.isclose</code> . <code>round()</code> = ties to even.
complex	<code>3+4j</code> , <code>.real</code> , <code>.imag</code> , <code>abs(z)</code> .
bool	True/False, subclass of int (<code>True + True == 2</code>).
str	Immutable text; indexing from 0 or -1; slicing <code>s[a:b:c]</code> ; f-strings.
list	Ordered, mutable, duplicates OK. <code>append</code> $O(1)$, <code>insert(0)</code> $O(n)$, <code>x in a</code> $O(n)$.
tuple	Ordered, immutable. <code>(5,)</code> for one item. Unpacking. Hashable if items are.
set	Unique, unordered, mutable. <code>set()</code> for empty. $O(1)$ in. <code> </code> & <code>-</code> <code>^</code> .
dict	key → value; keys unique + hashable; insertion-ordered; <code>get()</code> avoids <code>KeyError</code> .
None	Single "no value" object. Test with <code>is None</code> .

- `type(x)` → exact type. `isinstance(x, T)` → includes subclasses, accepts a tuple of types.

Type conversion, input, output

- Implicit only among numbers: `bool` → `int` → `float` → `complex`. `/` always returns float. `"5" + 5` is an error.
- Explicit: `int()` `float()` `str()` `bool()` `list()` `tuple()` `set()` `dict()`. `int("3.5")` fails; `int(3.9) = 3` (truncates); `bool("False")` is `True`.
- `input()` always returns `str`. Patterns: `int(input())`, `map(int, input().split())`, `list(map(int, input().split()))`.
- `print(*values, sep=" ", end="\n")`. `print` returns `None`. f-string formatting: `{x:.2f}`, `{x:05d}`, `{x:,}`.

Operators

- Arithmetic: + - * / // % **. // floors (-7//2 = -4). % has divisor's sign. ^ is XOR, not power. No ++/--.
- Comparison chains: 1 < x < 10. Different types with < → TypeError.
- Logical: and, or short-circuit and **return operands**; not returns a bool.
- Assignment: += etc. On lists += mutates in place, a = a + b makes a new list. Walrus :=.
- Membership in: substring for str, **keys** for dict; O(1) for set/dict, O(n) for list.
- == value equality; is identity. Use is only for None.
- Bitwise: & | ^ ~ << >>; n & 1 odd check; 1 << k mask; n & (n-1) clears lowest set bit; x << k = x * 2**k; ~x = -x-1.
- Precedence: () > ** > unary > * / // % > + - > shifts > & > ^ > | > comparisons/in/is > not > and > or.

Truthy and falsy

- Falsy: False None 0 0.0 0j "" [] () {} set() range(0). Everything else truthy ("0", "False", [0]).
- Use if arr: not if len(arr) > 0:. Use is None when 0 or empty are valid values.

11. Important Terminology

Term	Meaning
Python	High-level, interpreted, dynamically typed, general-purpose language
Interpreter	Program that reads and executes your code
Compiler	Program that translates the whole source into machine code before running
Bytecode	Low-level instructions Python compiles your code to (not machine code)
PVM	Python Virtual Machine: executes bytecode
CPython	The standard Python implementation, written in C
REPL	Read-Eval-Print Loop: the interactive >>> shell
High-level language	Language abstracted from hardware details
Dynamic typing	Types are attached to objects and checked at runtime
Strong typing	Incompatible types are not silently mixed ("5" + 5 fails)
Duck typing	What an object can do matters, not its class name
Variable / name	A label that refers to an object
Object	Data in memory with identity, type and value
Reference	The link from a name to an object
Identity	Unique ID of an object, <code>id(x)</code> ; checked by <code>is</code>
Binding / assignment	Making a name refer to an object
Alias	A second name for the same object
Mutable	Can be changed in place (list, dict, set)
Immutable	Cannot be changed after creation (int, str, tuple)
Hashable	Has a fixed hash for its life; usable as dict key / set element
Namespace	A mapping from names to objects (e.g. global, local)
Identifier	Name of a variable, function, class, etc.
Keyword	Reserved word with fixed meaning (if, for, def ...)
Docstring	Triple-quoted first string of a function/class/module, stored in <code>__doc__</code>
Indentation	Leading whitespace that defines a block
Statement	An instruction Python executes (<code>x = 5</code> , <code>if ...:</code>)
Expression	Code that produces a value (<code>2 + 3</code>)
Literal	A value written directly in code (<code>10</code> , <code>"hi"</code> , <code>[1, 2]</code>)
Operand / operator	The value / the symbol that acts on it
Unary / binary / ternary	Operator taking one / two / three operands
Precedence	Which operator is evaluated first
Associativity	Direction of evaluation for operators of equal precedence
Short-circuit	and/or stop as soon as the result is known
Truthy / falsy	Objects treated as True / False in conditions
Singleton	A class with only one object (e.g. None)
NoneType	The type of None
Sequence	Ordered, indexable collection (str, list, tuple, range)
Iterable	Anything you can loop over
Type casting	Explicit conversion with <code>int()</code> , <code>str()</code> ...
Coercion	Implicit automatic conversion (int → float)
Floor division	<code>//</code> : divide and round toward negative infinity

Term	Meaning
Modulus	%: remainder
Arbitrary precision	Ints can grow without limit
IEEE 754	Standard for floating-point numbers used by Python float
NaN / inf	Special floats: "not a number" / infinity
Slicing	Extracting part of a sequence, s[start:stop:step]
Unpacking	Assigning items of a sequence to several names
f-string	String with {} placeholders: f"{x:.2f}"
Escape sequence	Backslash code such as \n, \t
Augmented assignment	Shortcut like x += 1
Walrus operator	:= assigns inside an expression
Shadowing	Reusing a built-in's name (e.g. list = 5), hiding it
Reference counting	Freeing an object when nothing refers to it
Garbage collector	Frees unreachable objects, including cycles
Traceback	Error report showing where an exception happened
PEP 8	Official Python style guide
Bitmask	An integer used as a set of on/off flags

12. Twenty Conceptual Questions

Answer each question aloud or in writing **before** looking at the answers in Chapter 15.

- Q1.** What is Python? Is it compiled or interpreted?
- Q2.** Differentiate between an interpreter and a compiler.
- Q3.** What is dynamic typing? Is Python weakly typed?
- Q4.** What exactly happens when you write `x = 10`?
- Q5.** What are mutable and immutable objects? Give examples and explain why it matters.
- Q6.** What is the difference between `==` and `is`?
- Q7.** State the rules for identifiers. How do you see the keywords?
- Q8.** Why is indentation important in Python? What errors can occur?
- Q9.** Differentiate `type()` and `isinstance()`.
- Q10.** Why is `bool` a subclass of `int`? Give a consequence.
- Q11.** Compare list, tuple, set and dict.
- Q12.** What does "hashable" mean? Why can a tuple be a dict key but a list cannot?
- Q13.** Explain implicit and explicit type conversion with examples.
- Q14.** What does `input()` return? How do you read two integers from one line?
- Q15.** What are the `sep` and `end` parameters of `print()`? What does `print()` return?
- Q16.** Explain `/`, `//` and `%`, including negative numbers.
- Q17.** What is short-circuit evaluation? What do `and` / `or` return?
- Q18.** What are truthy and falsy values? List the falsy ones.
- Q19.** Why is `0.1 + 0.2 != 0.3`? How should floats be compared?
- Q20.** What is `None`? How do you test for it and which things return it?

🕒 INTERVIEW	Good interview answers are short, correct and end with one example. Practise saying them out loud in under 30 seconds.
--------------------	--

13. Twenty Output-Based Questions

Predict the output of each snippet (all run on Python 3.12). Write your prediction first, then check the answer in Chapter 15.

Q1.

```
x = 10
y = x
y += 5
print(x, y)
```

Q2.

```
a = [1, 2, 3]
b = a
b.append(4)
print(a, a is b)
```

Q3.

```
print(7 // 2, -7 // 2, 7 % -3, 2 ** 3 ** 2)
```

Q4.

```
print(type(5 / 2), type(5 // 2), type(5.0 // 2))
```

Q5.

```
print(True + True + 3, isinstance(True, int), type(True) == int)
```

Q6.

```
print(bool(""), bool(" "), bool("0"), bool([0]), bool(None))
```

Q7.

```
print(0.1 + 0.2 == 0.3, round(2.5), round(3.5), int(-3.9))
```

Q8.

```
print(3 and 4, 0 or 5, "" or None, not 0)
```

Q9.

```
t = (5)
u = (5,)
print(type(t), type(u), type({}), type(set()))
```

Q10.

```
a = [1, 2]
a += [3]
b = a
a = a + [4]
print(a, b)
```

Q11.

```
s = "PYTHON"
print(s[1:4], s[-2:], s[::-2], len(s))
```

Q12.

```
print(1 < 5 < 10, 1 < 5 > 10, 5 == 5.0, "a" < "B")
```

Q13.

```
x = 5
print(x is None, x == None, None == False, None is None)
```

Q14.

```
print(int("12") + int(3.9), "12" + str(3), "ab" * 2 + "c")
```

Q15.

```
print(5 & 3, 5 | 3, 5 ^ 3, ~5, 1 << 4, 20 >> 2)
```

Q16.

```
print(-2 ** 2, (-2) ** 2, not 1 == 2, 2 + 3 * 4 ** 2 / 8)
```

Q17.

```
a = b = []  
a.append(1)  
c = [[0] * 2] * 2  
c[0][0] = 9  
print(b, c)
```

Q18.

```
print(print("hi"))
```

Q19.

```
d = {"a": 1, "b": 2}  
print("a" in d, 1 in d, d.get("z"), d.get("z", 0), len(d))
```

Q20.

```
print(10 / 2, 2 ** -1, divmod(17, 5), 9 % 4 == 1)
```

14. Twenty Practice Problems

Write each program yourself. Then compare with the solutions in Chapter 15. Sample input is what would be typed on the keyboard.

Problem 1: Next-year age

Read a name and an age (two lines). Print: <name> will be <age+1> next year.

```
Sample input:
Riya
20
```

Problem 2: Six operations

Read two integers on one line. Print their sum, difference, product, true division, floor division and remainder, one per line.

```
Sample input:
17 5
```

Problem 3: Swap

Read two integers. Swap them (i) using tuple assignment and then (ii) using arithmetic only, without a temporary variable. Print after each swap.

```
Sample input:
3 8
```

Problem 4: Even or odd, two ways

Read an integer. Print Even or Odd using %, and again using the bitwise & operator.

```
Sample input:
7
```

Problem 5: Digit surgery

Read a positive integer. Print its last digit and the number with the last digit removed, without converting to a string.

```
Sample input:
4837
```

Problem 6: Circle

Read a radius (float). Print area and circumference, each with 2 decimal places (use `math.pi`).

```
Sample input:
2.5
```

Problem 7: Celsius to Fahrenheit

Read a temperature in Celsius (float). Print it in Fahrenheit with 1 decimal place. $F = C * 9/5 + 32$.

```
Sample input:
37
```

Problem 8: Seconds to hh:mm:ss

Read a number of seconds. Print it as hh:mm:ss with leading zeros. (Hint: `divmod`.)

```
Sample input:
3725
```

Problem 9: Type inspector

For the list `[5, 5.0, "5", True, None, [5], (5,), {5}, {"k": 5}, 5j]` print each value and the **name** of its type.

Problem 10: Largest of three

Read three integers. Print the largest using only comparison and logical operators (no `max()`).

```
Sample input:
12 45 9
```

Problem 11: Leap year

Read a year. Print Leap or Not Leap using one boolean expression. (Divisible by 4 but not by 100, unless divisible by 400.)

Sample input:
2100

Problem 12: Power of two

Read an integer. Print True if it is a power of two, else False, using a bitwise trick.

Sample input:
64

Problem 13: Bit toolkit

Read n and k. Print bit k of n (0 = rightmost), and n after setting, clearing and toggling bit k.

Sample input:
10 1

Problem 14: Alias or copy?

Create a = [1, 2, 3], b = a, c = a.copy(). Print a is b, a is c, a == c. Then b.append(4) and print a and c.

Problem 15: Float comparison

Show that 0.1 * 3 == 0.3 is False, and write a correct comparison using math.isclose and using abs.

Problem 16: Number systems

Read a binary string. Print its decimal value, and its binary, octal and hex forms.

Sample input:
1010

Problem 17: List statistics

Read integers on one line. Print sum, maximum, minimum and the count of even numbers.

Sample input:
4 7 1 10 3 8

Problem 18: Character frequency

Read a string. Print a dictionary of character counts using dict.get.

Sample input:
banana

Problem 19: FizzBuzz for one number

Read n. Print FizzBuzz if divisible by both 3 and 5, Fizz if only by 3, Buzz if only by 5, otherwise n itself.

Sample input:
30

Problem 20: Unique values

Read integers on one line. Print how many distinct values there are, the sorted distinct values, and whether any value repeats.

Sample input:
5 3 5 1 3 9

15. Answers and Solutions

15.1 Conceptual answers

Q1. What is Python? Is it compiled or interpreted?

Python is a high-level, general-purpose, dynamically typed language created by Guido van Rossum (1991). In CPython, source code is **compiled to bytecode**, and the bytecode is then **executed by the Python Virtual Machine**. So it is called an interpreted language, but strictly it does both steps.

Q2. Differentiate between an interpreter and a compiler.

A compiler translates the **whole** program into machine code before it runs and produces an executable (C, C++). An interpreter reads and executes the program **step by step** (classic Python). Compiled code usually runs faster; interpreted code is easier to test and debug and is more portable as source.

Q3. What is dynamic typing? Is Python weakly typed?

Dynamic typing means a variable has no declared type. The **type belongs to the object**, is checked at runtime, and the same name can refer to objects of different types. Python is **not weakly typed**: it is dynamically **and strongly** typed, because `"5" + 5` raises `TypeError` instead of silently converting.

Q4. What exactly happens when you write `x = 10`?

Python creates (or reuses) an int object with value 10, creates the name `x` in the current namespace and **binds** `x` to that object. A variable is a **name that references an object**, not a box holding a value. `id(x)` gives object identity, `type(x)` its type.

Q5. What are mutable and immutable objects? Give examples and explain why it matters.

Mutable objects can change in place (list, dict, set); immutable ones cannot (int, float, bool, str, tuple, frozenset). It matters because names can **share** a mutable object (`b = a`), so a change through one name is visible through the other, and because only hashable (immutable) objects can be dict keys or set elements.

Q6. What is the difference between `==` and `is`?

`==` compares **values**; `is` compares **identity** (same object, same id). `[1,2] == [1,2]` is True but `[1,2] is [1,2]` is False. Use `is` only for None (`x is None`). Integer/string caching can make `is` look like it works, but that is an implementation detail.

Q7. State the rules for identifiers. How do you see the keywords?

An identifier can contain letters, digits and `_`; it cannot start with a digit; it is case-sensitive; it cannot be a keyword; no spaces or special symbols. Keywords are listed by `import keyword; print(keyword.kwlist)` (35 in Python 3.12). True, False, None are the only capitalised ones.

Q8. Why is indentation important in Python? What errors can occur?

Indentation **defines blocks** (Python has no braces). Every block after a colon must be indented, with the same amount for all lines in the block (4 spaces by convention). Wrong indentation raises `IndentationError` (or `TabError` when tabs and spaces are mixed).

Q9. Differentiate `type()` and `isinstance()`.

`type(x)` returns the exact type. `isinstance(x, T)` returns True for `T` **or any subclass**, and accepts a tuple of types. Example: `isinstance(True, int)` is True but `type(True) == int` is False. Prefer `isinstance()` for checks.

Q10. Why is `bool` a subclass of `int`? Give a consequence.

For historical reasons True behaves as 1 and False as 0. So `True + True == 2`, `isinstance(True, int)` is True, and `sum(cond for x in data)` counts how many items satisfy a condition.

Q11. Compare list, tuple, set and dict.

list: ordered, mutable, allows duplicates. tuple: ordered, **immutable**, allows duplicates, hashable if items are. set: **unordered**, mutable, unique items only, O(1) average membership. dict: key → value mapping, keys unique and hashable, insertion-ordered since 3.7, O(1) average lookup by key.

Q12. What does "hashable" mean? Why can a tuple be a dict key but a list cannot?

A hashable object has a hash value that never changes during its lifetime (and supports ==). Immutable objects such as int, str and tuples of hashables qualify. Lists are mutable, so their hash could change while stored in a dict/set; therefore they are unhashable.

Q13. Explain implicit and explicit type conversion with examples.

Implicit: Python converts automatically in mixed **numeric** operations, in the order bool → int → float → complex (3 + 4.5 → 7.5). Explicit: you call int(), float(), str(), list() etc. (int("25")). Python never implicitly converts between str and numbers.

Q14. What does input() return? How do you read two integers from one line?

input() always returns a **string**. To read two integers: a, b = map(int, input().split()). input() gives "3 4", split() gives ["3", "4"], map(int, ...) converts each item, and tuple unpacking assigns them.

Q15. What are the sep and end parameters of print()? What does print() return?

sep (default " ") goes between values; end (default "\n") goes after the last value. print() returns None, so x = print("hi") stores None. Printing shows a value; returning hands it back to the caller.

Q16. Explain /, // and %, including negative numbers.

/ is true division and always returns a float. // is floor division (rounds toward negative infinity): -7 // 2 = -4. % is the remainder with the **sign of the divisor**: -7 % 3 = 2. They satisfy a == (a // b) * b + a % b.

Q17. What is short-circuit evaluation? What do and / or return?

and/or stop evaluating once the result is known and return **one of the operands**. a and b returns a if it is falsy else b; a or b returns a if it is truthy else b. So x or "default" is a default-value idiom, and arr and arr[0] avoids an IndexError on an empty list.

Q18. What are truthy and falsy values? List the falsy ones.

In conditions every object is treated as true-like or false-like. Falsy: False, None, 0, 0.0, 0j, "", [], (), {}, set(), range(). Everything else is truthy, including "0", "False" and [0].

Q19. Why is 0.1 + 0.2 != 0.3? How should floats be compared?

Floats are stored in binary (IEEE 754). Numbers like 0.1 cannot be represented exactly, so tiny rounding errors appear (0.30000000000000004). Compare with math.isclose(a, b) or abs(a - b) < 1e-9, never with ==.

Q20. What is None? How do you test for it and which things return it?

None is the single object of type NoneType meaning "no value". Test with x is None. It is returned by functions without a return, by print(), and by in-place methods such as list.append, list.sort, list.reverse.

15.2 Output-based answers

Q1. Ints are immutable. y += 5 rebinds y to a **new** object 15; x still points to 10.

```
Output:
10 15
```

Q2. b = a creates an alias. append mutates the shared list, so a shows the change and a is b is True.

```
Output:
[1, 2, 3, 4] True
```

Q3. -7 // 2 floors to -4. 7 % -3 takes the sign of the divisor: -2. ** is right-associative: 2 ** (3 ** 2) = 512.

```
Output:
3 -4 -2 512
```

Q4. / always gives a float. // on two ints gives an int; if either operand is a float, the result is a float.

Output:
<class 'float'> <class 'int'> <class 'float'>

Q5. True is 1, so $1 + 1 + 3 = 5$. bool is a subclass of int, so isinstance is True; but the exact type is bool, so the last comparison is False.

Output:
5 True False

Q6. Only the empty string and None are falsy. " ", "0" and [0] are non-empty, so truthy.

Output:
False True True True False

Q7. Float rounding error makes the first False. round uses banker's rounding (ties go to the even number): 2 and 4. int() truncates toward zero: -3.

Output:
False 2 4 -3

Q8. 3 and 4 → both truthy, returns the last (4). 0 or 5 → 5. "" or None → both falsy, returns the last (None). not 0 → True.

Output:
4 5 None True

Q9. (5) is just an int in brackets; the comma makes a tuple. {} creates an empty **dict**; use set() for an empty set.

Output:
<class 'int'> <class 'tuple'> <class 'dict'> <class 'set'>

Q10. a += [3] mutates in place. Then b aliases that list. a = a + [4] builds a **new** list and rebinds a; b keeps the old one.

Output:
[1, 2, 3, 4] [1, 2, 3]

Q11. s[1:4] = indexes 1,2,3. s[-2:] = last two. s[::-2] starts at the end and steps back by 2: indexes 5, 3, 1 = N, H, Y.

Output:
YTH ON NHY 6

Q12. Chained comparisons: $1 < 5$ and $5 < 10$ is True; $1 < 5$ and $5 > 10$ is False. $5 == 5.0$ is True. Strings compare by Unicode code: "a" is 97, "B" is 66, so "a" < "B" is False.

Output:
True False True False

Q13. None equals only None. It is falsy but **not equal** to False. None is None is True because there is a single None object.

Output:
False False False True

Q14. int("12") is 12 and int(3.9) is 3, so 15. The second is string concatenation. "ab" * 2 is "abab", then + "c".

Output:
15 123 ababc

Q15. $101 \& 011 = 001$ (1); OR = 111 (7); XOR = 110 (6); $\sim 5 = -(5+1) = -6$; $1 \ll 4 = 16$; $20 \gg 2 = 20 // 4 = 5$.

Output:
1 7 6 -6 16 5

Q16. ** binds tighter than unary minus: $-(2**2) = -4$. not (1 == 2) is True. Last: $4**2 = 16$, $3*16 = 48$, $48/8 = 6.0$, $2 + 6.0 = 8.0$.

Output:
-4 4 True 8.0

Q17. Chained assignment makes a and b the same list. `[[0]*2]*2` repeats a **reference** to one inner list, so changing `c[0][0]` changes both rows.

Output:
`[1] [[9, 0], [9, 0]]`

Q18. The inner print shows hi and returns None. The outer print then prints that None.

Output:
hi
None

Q19. `in` checks **keys**. 1 is a value, not a key. `get` returns None (or the default) for a missing key instead of raising `KeyError`.

Output:
True False None 0 2

Q20. `10 / 2` is 5.0. A negative exponent gives a float: 0.5. `divmod(17, 5)` returns (3, 2). `9 % 4` is 1, so True.

Output:
5.0 0.5 (3, 2) True

15.3 Practice problem solutions

Problem 1: Next-year age

```
name = input()
age = int(input())
print(f"{name} will be {age + 1} next year")
```

Keyboard input:
Riya
20
Output:
Riya will be 21 next year

Problem 2: Six operations

```
a, b = map(int, input().split())
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
```

Keyboard input:
17 5
Output:
22
12
85
3.4
3
2

Problem 3: Swap

```
a, b = map(int, input().split())
a, b = b, a
print(a, b)
# arithmetic swap (swaps them back)
a = a + b
b = a - b
a = a - b
print(a, b)
```

Keyboard input:
3 8
Output:
8 3
3 8

Problem 4: Even or odd, two ways

```
n = int(input())
print("Even" if n % 2 == 0 else "Odd")
print("Even" if (n & 1) == 0 else "Odd")
```

Keyboard input:

7

Output:

Odd

Odd

Problem 5: Digit surgery

```
n = int(input())
print(n % 10)
print(n // 10)
```

Keyboard input:

4837

Output:

7

483

Problem 6: Circle

```
import math
r = float(input())
print(f"Area: {math.pi * r * r:.2f}")
print(f"Circumference: {2 * math.pi * r:.2f}")
```

Keyboard input:

2.5

Output:

Area: 19.63

Circumference: 15.71

Problem 7: Celsius to Fahrenheit

```
c = float(input())
f = c * 9 / 5 + 32
print(f"{f:.1f}")
```

Keyboard input:

37

Output:

98.6

Problem 8: Seconds to hh:mm:ss

```
s = int(input())
m, sec = divmod(s, 60)
h, m = divmod(m, 60)
print(f"{h:02d}:{m:02d}:{sec:02d}")
```

Keyboard input:

3725

Output:

01:02:05

Problem 9: Type inspector

```
values = [5, 5.0, "5", True, None, [5], (5,), {5}, {"k": 5}, 5j]
for v in values:
    print(repr(v), "->", type(v).__name__)
```

Output:

```
5 -> int
5.0 -> float
'5' -> str
True -> bool
None -> NoneType
[5] -> list
(5,) -> tuple
{5} -> set
{'k': 5} -> dict
5j -> complex
```

Problem 10: Largest of three

```
a, b, c = map(int, input().split())
if a >= b and a >= c:
    print(a)
elif b >= a and b >= c:
    print(b)
else:
    print(c)
```

Keyboard input:

12 45 9

Output:

45

Problem 11: Leap year

```
year = int(input())
leap = (year % 4 == 0 and year % 100 != 0) or year % 400 == 0
print("Leap" if leap else "Not leap")
```

Keyboard input:

2100

Output:

Not leap

Problem 12: Power of two

```
n = int(input())
print(n > 0 and (n & (n - 1)) == 0)
```

Keyboard input:

64

Output:

True

Problem 13: Bit toolkit

```
n, k = map(int, input().split())
print((n >> k) & 1)
print(n | (1 << k))
print(n & ~(1 << k))
print(n ^ (1 << k))
```

Keyboard input:

10 1

Output:

```
1
10
8
8
```

Problem 14: Alias or copy?

```
a = [1, 2, 3]
b = a
c = a.copy()
print(a is b, a is c, a == c)
b.append(4)
print(a)
print(c)
```

Output:

```
True False True
[1, 2, 3, 4]
[1, 2, 3]
```

Problem 15: Float comparison

```
import math
x = 0.1 * 3
print(x, x == 0.3)
print(math.isclose(x, 0.3))
print(abs(x - 0.3) < 1e-9)
```

Output:

```
0.30000000000000004 False
True
True
```

Problem 16: Number systems

```
s = input()
n = int(s, 2)
print(n, bin(n), oct(n), hex(n))
```

Keyboard input:

```
1010
```

Output:

```
10 0b1010 0o12 0xa
```

Problem 17: List statistics

```
arr = list(map(int, input().split()))
print(sum(arr), max(arr), min(arr))
print(sum(x % 2 == 0 for x in arr))
```

Keyboard input:

```
4 7 1 10 3 8
```

Output:

```
33 10 1
3
```

Problem 18: Character frequency

```
s = input()
freq = {}
for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

Keyboard input:

```
banana
```

Output:

```
{'b': 1, 'a': 3, 'n': 2}
```

Problem 19: FizzBuzz for one number

```
n = int(input())
if n % 15 == 0:
    print("FizzBuzz")
elif n % 3 == 0:
    print("Fizz")
elif n % 5 == 0:
    print("Buzz")
else:
    print(n)
```

Keyboard input:

30

Output:

FizzBuzz

Problem 20: Unique values

```
arr = list(map(int, input().split()))
u = set(arr)
print(len(u), sorted(u))
print(len(u) != len(arr))
```

Keyboard input:

5 3 5 1 3 9

Output:

4 [1, 3, 5, 9]

True

16. One-Page Cheat Sheet: Part 1

1. Types at a glance

Type	Mut	Ord	Dup	Notes
int/float	no	-	-	int unlimited; float approx
bool	no	-	-	subclass of int
str	no	yes	yes	indexed, sliceable
list []	yes	yes	yes	append O(1), in O(n)
tuple ()	no	yes	yes	(5,), hashable
set set()	yes	no	no	in O(1), no index
dict {}	yes	yes*	keys no	*insertion order; get
None	no	-	-	use is None

2. Conversion

Implicit	bool→int→float→complex; / always float
Explicit	int() float() str() bool() list() tuple() set() dict()
Traps	int("3.5") error; int(-3.9)=-3; bool("0")=True; "5"+5 error

3. Input / Output

n = int(input())	one int
a, b = map(int, input().split())	two ints, one line
arr = list(map(int, input().split()))	list of ints
print(a, b, sep="-", end="")	separator / ending
print(*arr)	space separated list
f"{x:.2f} {n:05d} {n:,}"	format spec
input() → str always; print() → None	

4. Arithmetic

/	float: 7/2=3.5
//	floor: 7//2=3, -7//2=-4
%	sign of divisor: -7%3=2
**	right-assoc: 2**3**2=512; -2**2=-4
^	XOR, NOT power
no ++ / --	use += 1

5. Logical & truthiness

a and b	a if a falsy else b
a or b	a if a truthy else b
not a	always bool
Falsy	False None 0 0.0 0j "" [] () {} set() range(0)
Truthy traps	"0" "False" [0] " " are all truthy

6. Membership & identity

in	str: substring; dict: KEYS; set/dict O(1), list O(n)
==	same value ([1]==[1] True)
is	same object ([1] is [1] False); use for None only

7. Precedence (high → low)

1	()
2	** (right to left)
3	unary +x -x ~x
4	* / // %
5	+ -
6	<< >>
7	&
8	^
9	
10	== != < > <= >= is in (chained)
11	not
12	and
13	or
14	x if c else y, lambda

8. Bitwise

& ^	AND, OR, XOR
~x	-x-1
x<<k, x>>k	x*2**k, x//2**k
odd?	n & 1
bit k	(n >> k) & 1
set / clear / toggle	n (1<<k) n&~(1<<k) n^(1<<k)
power of 2	n>0 and n&(n-1)==0
unique number	XOR everything: x^x=0

9. Identifiers, syntax

Identifier	letters/digits/_, no leading digit, case-sensitive, not keyword
Style	snake_case PascalCase UPPER_CASE _private
Blocks	colon + 4 spaces; never mix tabs/spaces; pass = empty block
Comments	#; docstring = first triple-quoted string
Keywords	35; only True/False/None capitalised

10. Memory & mutability

Name → object	assignment binds, never copies; b = a = alias
Immutable	int float complex bool str tuple frozenset None
Mutable	list dict set
Copy	a.copy(), a[:]; deep: copy.deepcopy
Cache	small ints -5..256; never rely on is for values

11. type() vs isinstance()

type(x)	exact type; type(True)==int False
isinstance(x, T)	subclasses OK; tuple of types; isinstance(True,int) True

12. Top mistakes

= vs ==	n = input() is str	x is 5
{ } is a dict	(5) is not a tuple	[[0]*3]*3
a = a.sort()	x == 1 or 2	0.1+0.2 == 0.3